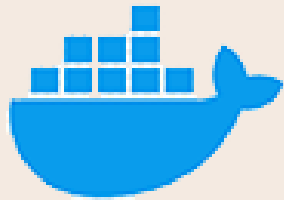


Docker&Kubernetes

컨테이너/이미지 다뤄 보기



≡ 컨테이너 실행해 보기

≡ 컨테이너를 생성하고 실행해보기

다음과 같은 이미지 정보를 사용하여 요구 사항에 맞는 컨테이너를 생성해 보도록 합니다.

설치할 컨테이너

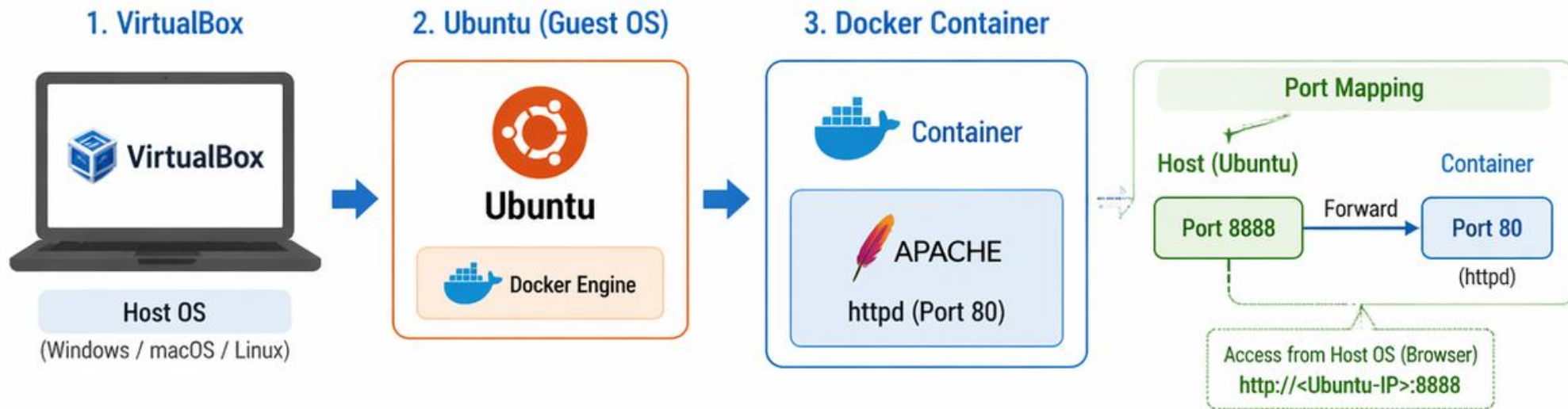
항목	설명
httpd	Apache 웹 서버를 실행하기 위한 Docker 이미지입니다. 웹 서버를 컨테이너화하여 간단히 애플리케이션 배포와 테스트를 할 수 있습니다. 오픈 소스 HTTP 서버로, 웹 페이지를 호스팅하는 데 널리 사용됩니다.
nginx	웹 서버 설정 및 배포. 로드 밸런싱 및 리버스 프록시 서버 구축.

≡ 컨테이너 실행해 보기

≡ 작업하고자 하는 내용

작업하고자 하는 내용은 다음과 같습니다.

Run Apache Web Server in a Docker Container on Ubuntu (VirtualBox)



≡ 컨테이너 실행해 보기

≡ 작업하고자 하는 내용

작업하고자 하는 내용은 다음과 같습니다.

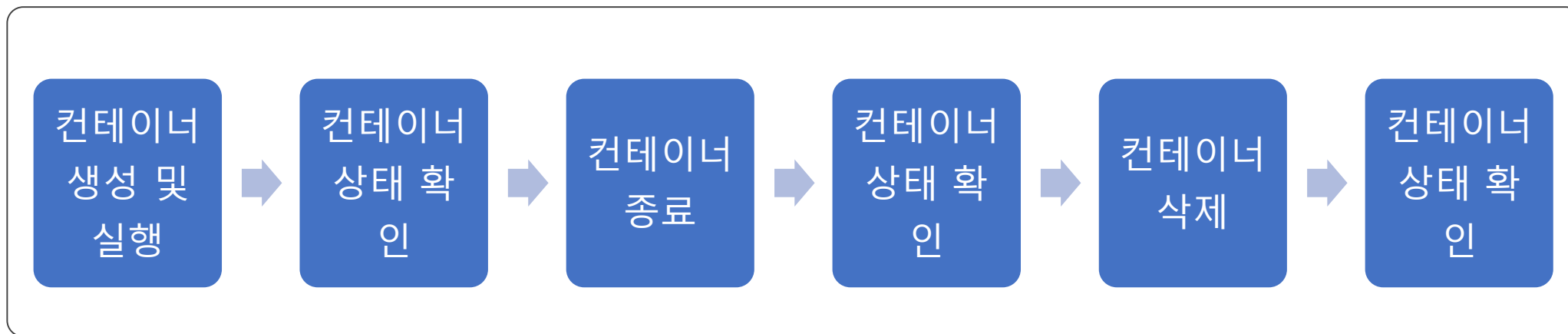
구현할 내용

웹 서버 기능을 제공해 주는 apache 컨테이너를 생성하고, 실행 및 상태를 확인합니다.

해당 포트를 이용하여 브라우저에서 접속이 잘 되는 지 점검합니다.

이후 컨테이너를 종료 및 삭제 테스트도 진행해 보도록 합니다.

참고로 apache의 이미지 이름은 'httpd'입니다.



≡ 컨테이너 실행해 보기

≡ 컨테이너 생성 및 실행, 상태 확인, 종료, 삭제

다음과 같이 테스트를 진행해 보도록 합니다.

키워드는 윈도우 cmd의 주석 구문입니다. 리눅스에서는 # 기호를 사용하면 됩니다.

컨테이너 생성 : 내 컴퓨터의 포트 번호(8888), 컨테이너 내부 포트 번호(80)

```
docker container run --name apache01 -d -p 8888:80 httpd
```

도커 이미지가 없는 경우에 최초 "Unable to..."라는 메시지를 보여 주고, 도커 허브에서 로컬로 이미지를 다운로드 합니다.

Unable to find image 'httpd:latest' locally

latest: Pulling from library/httpd

Digest: sha256:f4c5139eda466e45814122d9bd8b886d8ef6877296126c09b76dbad72b03c336

Status: Downloaded newer image for httpd:latest

실행 중인 컨테이너의 목록을 확인합니다.

```
docker container ls
```

STATUS 컬럼에 'Up'이라는 문장으로 실행되고 있으면 해당 컨테이너가 실행 중이라는 뜻입니다.

0.0.0.0는 모든 IPv4 주소를 대표하는 값 의미합니다.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e0b24775004a	httpd	"httpd-foreground"	About a minute ago	Up About a minute	0.0.0.0:8888->80/tcp	apache01

≡ 컨테이너 실행해 보기

≡ 컨테이너 생성 및 실행, 상태 확인, 종료, 삭제

다음과 같이 테스트를 진행해 보도록 합니다.

모든 컨테이너의 목록을 확인합니다.

`docker container ps -a`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e0b24775004a	httpd	"httpd-foreground"	2 minutes ago	Up 2 minutes	0.0.0.0:8888->80/tcp	apache01

docker.desktop

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	 apache01	e0b24775004a	httpd	8888:80 ↗	0.01%	3 minutes ago	  

≡ 컨테이너 실행해 보기

≡ 컨테이너 생성 및 실행, 상태 확인, 종료, 삭제

다음과 같이 테스트를 진행해 보도록 합니다.

Nat 모드

브라우저에 다음 주소를 입력하여 화면과 같이 동일하게 출력이 되는지 점검합니다

윈도우인 경우에는 접속이 가능하나, 리눅스를 사용하면 접속이 되지 않습니다.

<http://localhost:8888>

접속이 안됩니다.(nat 설정 필요)

Bridge 모드

우선 IP 주소를 확인합니다

`ip addr`

enp0s3 영역의 inet에 있는 IP 주소를 확인합니다.

예시) 220.73.38.125

☰ 컨테이너 실행해 보기

☰ 컨테이너 생성 및 실행, 상태 확인, 종료, 삭제

for use only Linux

Nat 모드인 경우 윈도우에서 리눅스로 접속이 가능하도록 다음과 같이 설정해야 합니다.

The screenshot shows the Oracle VM VirtualBox Manager interface. On the left, a list of VMs is shown: 'MyUbuntu' (powered off), 'FullStack' (powered off), and 'Broadcast' (running). The 'Broadcast' VM is selected, and its settings are displayed on the right. The 'Network' tab is active, showing the 'NAT 네트워크' (NAT Network) configuration. The 'BroadcastNAT' network is listed with the following details:

이름	IPv4 접두사	IPv6 접두사	DHCP 서버
BroadcastNAT	10.0.2.0/24	fd17:625c:f037:2::/64	사용함

Below this, the '포트 포워딩(P)' (Port Forwarding) tab is selected. It shows a table of port forwarding rules. A red rectangle highlights the first two rules:

이름	프로토콜	호스트 IP	호스트 포트	게스트 IP	게스트 포트
ApacheTest	TCP		8888	10.0.2.4	8888
Broadcast_HTTP	TCP		80	10.0.2.4	80

At the bottom right of the settings window, there are buttons for '적용' (Apply) and '초기화' (Reset).

≡ 컨테이너 실행해 보기

≡ 컨테이너 생성 및 실행, 상태 확인, 종료, 삭제

다음과 같이 테스트를 진행해 보도록 합니다.

Nat 모드

브라우저에 다음 주소를 입력하여 화면과 같이 동일하게 출력이 되는지 점검합니다

<http://localhost:8888>

It works!

Bridge 모드

브라우저에 다음과 같이 IP 주소 뒤에 포트 번호를 붙여서 출력이 되는지 점검합니다

<http://220.73.38.125:8888>

It works!

≡ 컨테이너 실행해 보기

≡ 컨테이너 생성 및 실행, 상태 확인, 종료, 삭제

다음과 같이 테스트를 진행해 보도록 합니다.

stop 명령어를 사용하여 해당 컨테이너를 종료 하도록 합니다.

```
docker container stop apache01
```

```
apache01
```

모든 컨테이너의 목록을 확인합니다.

```
docker container ps -a
```

STATUS 컬럼에 'Exited'이라는 문장으로 실행되고 있으면 해당 컨테이너가 '종료' 상태를 의미합니다.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e0b24775004a	httpd	"httpd-foreground"	9 minutes ago	Exited (0) 8 seconds ago		apache01

docker.desktop

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	 apache01	d0bb29a68113	httpd		N/A	25 minutes ago	  

≡ 컨테이너 실행해 보기

≡ 컨테이너 생성 및 실행, 상태 확인, 종료, 삭제

다음과 같이 테스트를 진행해 보도록 합니다.

rm 명령어를 사용하여 해당 컨테이너를 삭제 하도록 합니다.

```
docker container rm apache01
```

모든 컨테이너의 목록을 확인합니다.

```
docker container ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

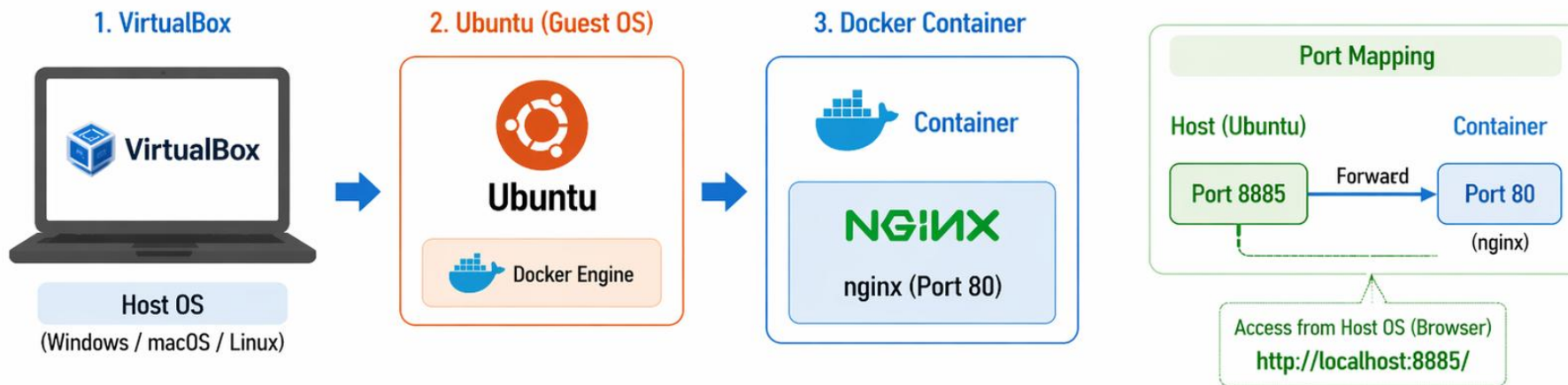
☰ 컨테이너 실행해 보기

☰ 컨테이너 생성 및 실행, 상태 확인, 종료, 삭제

풀어 봅시다.

다음 요구 사항대로 문제를 풀어 보세요.

Run Nginx Web Server in a Docker Container on Ubuntu (VirtualBox)



Requirements

- Web Server : nginx (Container)
- Image Name : nginx
- Container Name : nginx85
- Port : 8885
- Test URL : <http://localhost:8885/>

Test Result (Access in Browser)

← → ↻ ⓘ <http://localhost:8885/>

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and configured. Further configuration is required.

For online documentation and support please refer to nginx.org. Commercial support is available at nginx.com.

Thank you for using nginx.

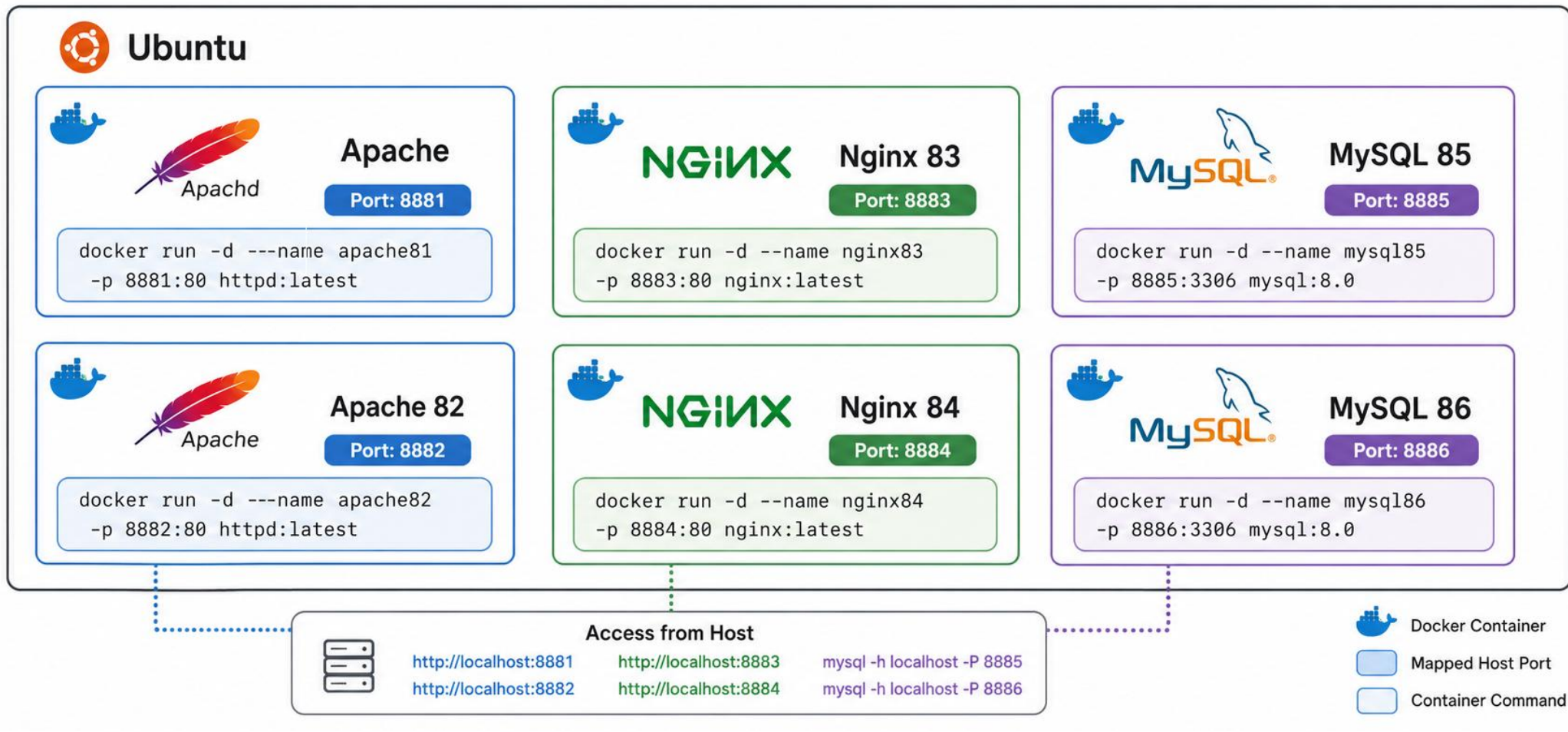
```
docker container run --name nginx85 -d -p 8885:80 nginx
docker container ps -a
http://localhost:8885/
docker container stop nginx85
docker container ps -a
docker container rm nginx85
```

☰ 여러 개의 컨테이너 생성

☰ 아파치 컨테이너 여러 개 생성하기

다음과 같이 테스트를 진행해 보도록 합니다.

Running Multiple Docker Containers on Ubuntu



≡ 여러 개의 컨테이너 생성

≡ 아파치 컨테이너 여러 개 생성하기

다음과 같이 테스트를 진행해 보도록 합니다.

설치할 컨테이너

항목	설명
httpd	Apache 웹 서버를 실행하기 위한 Docker 이미지입니다. 웹 서버를 컨테이너화하여 간단히 애플리케이션 배포와 테스트를 할 수 있습니다. 오픈 소스 HTTP 서버로, 웹 페이지를 호스팅하는 데 널리 사용됩니다.
nginx	웹 서버 설정 및 배포. 로드 밸런싱 및 리버스 프록시 서버 구축.
mysql	데이터베이스 서버 환경 구축에 필요한 컨테이너입니다. 개발 및 테스트용 데이터베이스를 제공합니다.

생성할 컨테이너 정보

컨테이너 이름	이미지 이름	포트 설정	루트 패스워드
apache81	httpd	8881:80	
apache82	httpd	8882:80	
nginx83	nginx	8883:80	
nginx84	nginx	8884:80	
mysql85	mysql	8885:80	root
mysql86	mysql	8886:80	root

≡ 여러 개의 컨테이너 생성

≡ 아파치 컨테이너 여러 개 생성하기

다음과 같이 테스트를 진행해 보도록 합니다.

```
# 다음과 같이 아파치와 Nginx 컨테이너를 여러 개 생성해 보도록 하겠습니다.  
# 컨테이너를 여러 개 생성할 때는 포트 번호를 중복하며 안되며, 컨테이너 포트 번호는 동일해도 상관이 없습니다.  
docker container run --name apache81 -d -p 8881:80 httpd  
docker container run --name apache82 -d -p 8882:80 httpd  
  
docker container run --name nginx83 -d -p 8883:80 nginx  
docker container run --name nginx84 -d -p 8884:80 nginx  
  
# 백그라운드에서 실행 및 키보드를 사용하여 컨테이너 내부의 파일 시스템을 조작합니다.  
# -e는 "환경 변수(Environment Variable) 설정 옵션"을 의미합니다.  
docker container run --name=mysql85 -dit -e MYSQL_ROOT_PASSWORD=root mysql  
docker container run --name=mysql86 -dit -e MYSQL_ROOT_PASSWORD=root mysql
```

☰ 여러 개의 컨테이너 생성

☰ 아파치 컨테이너 여러 개 생성하기

다음과 같이 테스트를 진행해 보도록 합니다.

ps 커맨드를 사용하여 컨테이너가 실행중인지 확인합니다.

컬럼 STATUS, PORTS를 유심히 확인해 봅니다.

`docker container ps`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
afb3c06b35dc	httpd	"httpd-foreground"	3 minutes ago	Up 3 minutes	0.0.0.0:8881->80/tcp, [::]:8881->80/tcp	apache81
b79729bf8cb2	httpd	"httpd-foreground"	3 minutes ago	Up 3 minutes	0.0.0.0:8882->80/tcp, [::]:8882->80/tcp	apache82
fa313b8af532	nginx	"/docker-entrypoint.⋯"	2 minutes ago	Up 2 minutes	0.0.0.0:8883->80/tcp, [::]:8883->80/tcp	nginx83
8341e6896caa	nginx	"/docker-entrypoint.⋯"	2 minutes ago	Up 2 minutes	0.0.0.0:8884->80/tcp, [::]:8884->80/tcp	nginx84
b15b063fec20	mysql	"docker-entrypoint.s⋯"	About a minute ago	Up About a minute	3306/tcp, 33060/tcp	mysql85
c10ab4d5021a	mysql	"docker-entrypoint.s⋯"	27 seconds ago	Up 26 seconds	3306/tcp, 33060/tcp	mysql86

☰ 여러 개의 컨테이너 생성

☰ 아파치 컨테이너 여러 개 생성하기

다음과 같이 테스트를 진행해 보도록 합니다.

브라우저에 다음 주소를 각각 입력하여 화면과 같이 동일하게 출력이 되는지 점검합니다

<http://localhost:8881> 또는 <http://xx.xx.xx.xx:8881>

<http://localhost:8882>

<http://localhost:8883>

<http://localhost:8884>

윈도우에서는 잘 접속이 되고, 리눅스에서는 제대로 동작하지 않습니다.

It works!

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

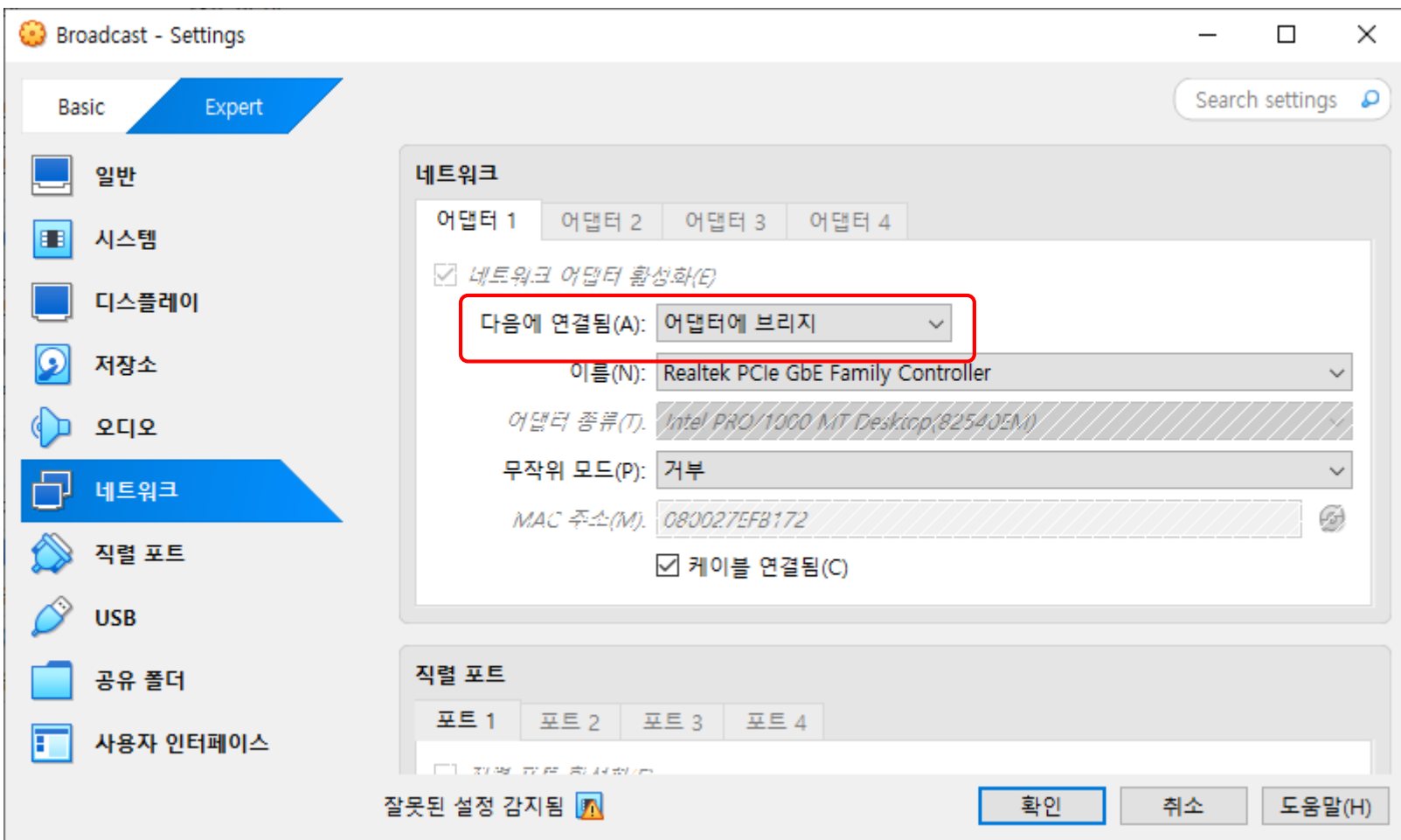
여러 개의 컨테이너 생성

아파치 컨테이너 여러 개 생성하기

for use only Linux

여러 개의 포트를 접속하고자 하는 경우 `Bridge`(`브리지`) 설정을 하는 것이 편리합니다.

부팅 중인 리눅스를 우선 종료하도록 합니다.



☰ 여러 개의 컨테이너 생성

☰ 아파치 컨테이너 여러 개 생성하기

for use only Linux

여러 개의 포트를 접속하고자 하는 경우 Bridge 설정을 하는 것이 편리합니다.

확인 사항

IP 주소 확인

`ip addr show`

ip a 명령어도 가능함

보통 ens33, eth0, `enp0s3` 등 인터페이스 아래에서 IP가 나옵니다.

XX.XX.XX.XX # 이 IP 주소는 재부팅시 변경이 됩니다.

위의 IP 주소를 사용하여 MobaXTerm을 사용하여 접속을 합니다.

≡ 여러 개의 컨테이너 생성

≡ 아파치 컨테이너 여러 개 생성하기

Bridge 설정으로 접속을 했다면 다음과 같이 접속 테스트를 수행해야 합니다.

도커가 멈추어 있다면 다음과 같이 동작시키도록 합니다.

`docker container start apache81`

브라우저에 다음 주소를 각각 입력하여 화면과 같이 동일하게 출력이 되는지 점검합니다

<http://http://xx.xx.xx.xx/:8881>

<http://http://xx.xx.xx.xx/:8882>

<http://http://xx.xx.xx.xx/:8883>

<http://http://xx.xx.xx.xx/:8884>

It works!

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

☰ 여러 개의 컨테이너 생성

☰ 컨테이너 파일 수정하기

Apache 컨테이너로 직접 들어가서 다음과 같이 파일을 작성하면 됩니다.

```
docker exec -it apache81 /bin/bash
```

```
apt update
```

```
apt install -y vim # vi 편집기 설치하기
```

```
cd /usr/local/apache2/htdocs/
```

```
vi index.html
```

```
# "hello apache 01!"라고 수정한 다음 다음과 같이 빠져 나가기 :wq
```

```
exit # 도커 빠져 나가기
```

<http://xx.xx.xx.xx:8881>로 다시 접속해보기

hello apache 01!

≡ 여러 개의 컨테이너 생성

≡ 내가 만든 파일로 대체하기

개발자가 직접 구현해 놓은 파일을 이용하여 나만의 페이지를 만들어 봅니다.

```
cd # 홈 디렉토리로 이동
# 나만의 홈페이지 파일(index.html) 교체
git clone https://github.com/seoljinuk/ec2_homepage.git

cd ec2_homepage\Public-2A

docker cp index.html apache82:/usr/local/apache2/htdocs/index.html
Successfully copied 2.56kB to apache82:/usr/local/apache2/htdocs/index.html

docker cp myimage01.png apache82:/usr/local/apache2/htdocs/myimage01.png
Successfully copied 2.56kB to apache82:/usr/local/apache2/htdocs/myimage01.png

http://localhost:8882로 접속을 시도해 봅니다.
```

웹 서버 01번

첫 번째 인스턴스 테스트 페이지입니다.



≡ 여러 개의 컨테이너 생성

≡ 데이터 베이스 접속

데이터 베이스 컨테이너가 정상적으로 동작하는 지 확인해 봅니다.

다음 문장을 실행시 "ready for connections...."라는 문구가 나오면 성공이라고 보면 됩니다.

`docker logs mysql85 | grep ready`

2026-04-25T07:52:52.819868Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: **ready for connections**. Version: '9.7.0' socket:
'/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.

2026-04-25T07:52:52.822075Z 0 [System] [MY-011323] [Server] X Plugin **ready for connections**. Bind-address: '::' port: 33060, socket:
/var/run/mysqld/mysqldx.sock

≡ 여러 개의 컨테이너 생성

≡ 데이터 베이스 접속

데이터 베이스 컨테이너가 정상적으로 동작하는 지 확인해 봅니다.

```
docker exec -it mysql85 bash
```

```
bash-5.1# mysql -u root -p
```

```
Enter password: root 입력(눈에 보이지 않음)
```

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
```

```
Your MySQL connection id is 9
```

```
Server version: 9.7.0 MySQL Community Server - GPL
```

```
Copyright (c) 2000, 2026, Oracle and/or its affiliates.
```

```
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql>
```


≡ 컨테이너 실행해 보기

≡ 컨테이너 파일 수정하기

풀어 봅시다.

nginx 컨테이너로 직접 들어가서 다음과 같이 파일을 작성하면 됩니다.

```
docker exec -it nginx83 /bin/bash
```

```
apt update
```

```
apt install -y vim
```

```
cd /usr/share/nginx/html/
```

```
vi index.html
```

```
# "Welcome"을 "Hello"라고 수정한 다음 다음과 같이 빠져 나가기 :wq
```

```
exit # 도커 빠져 나가기
```

<http://localhost:8883>로 접속을 시도해 봅시다.

Hello to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

≡ 여러 개의 컨테이너 생성

≡ 내가 만든 파일로 대체하기

풀어 봅시다.

개발자가 직접 구현해 놓은 파일을 이용하여 나만의 페이지를 만들어 봅니다.

```
cd
```

```
cd ec2_homepage\Public-2C
```

```
docker cp index.html nginx84:/usr/share/nginx/html/index.html
```

```
Successfully copied 2.56kB to nginx84:/usr/share/nginx/html/index.html
```

```
docker cp myimage02.png nginx84:/usr/share/nginx/html/myimage02.png
```

```
Successfully copied 2.56kB to nginx84:/usr/share/nginx/html/myimage02.png
```

<http://localhost:8884>로 접속을 시도해 봅니다.

웹 서버 02번

두 번째 인스턴스 테스트 페이지입니다.



≡ 여러 개의 컨테이너 생성

≡ 아파치 컨테이너 여러 개 생성하기

다음과 같이 테스트를 진행해 보도록 합니다.

stop 커맨드를 사용하여 컨테이너를 종료합니다.

```
docker container stop apache81 apache82 nginx83 nginx84 mysql85 mysql86
```

apache81

apache82

nginx83

nginx84

mysql85

mysql86

rm 커맨드를 사용하여 컨테이너를 삭제합니다.

```
docker container rm apache81 apache82 nginx83 nginx84 mysql85 mysql86
```

apache81

apache82

nginx83

nginx84

mysql85

mysql86

☰ 여러 개의 컨테이너 생성

☰ 아파치 컨테이너 여러 개 생성하기

다음과 같이 테스트를 진행해 보도록 합니다.

ps 커맨드와 인자를 사용하여 컨테이너가 잘 삭제되었는 지 확인합니다.

`docker container ps -a`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

≡ 이미지 삭제하기

≡ 컨테이너에 사용한 이미지 삭제하기

다음과 같이 테스트를 진행해 보도록 합니다.

docker image로부터 생성된 Container를 모두 삭제했습니다.

하지만 Container를 삭제하더라도 이미지 정보는 계속 존재합니다.

docker image 정보를 확인합니다.

이미지의 이름 이외에도 태그(버전 정보), 이미지 ID 등의 정보를 확인할 수 있습니다.

`docker image ls` # 도커의 이미지 목록 정보를 확인합니다.

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	fb197595ebe7	13 days ago	278MB
mysql	latest	0255b469f013	8 weeks ago	828MB
httpd	latest	f4c5139eda46	4 months ago	220MB

≡ 이미지 삭제하기

≡ 컨테이너에 사용한 이미지 삭제하기

다음과 같이 테스트를 진행해 보도록 합니다.

```
# rm 명령어를 사용하여 이미지 삭제가 가능합니다.  
# 이미지 이름 또는 이미지 ID를 이용하여 삭제하도록 합니다.  
docker image rm nginx mysql httpd  
Untagged: httpd:latest  
Deleted: sha256:f4c5139eda466e45814122d9bd8b886d8ef6877296126c09b76dbad72b03c336  
Untagged: mysql:latest  
Deleted: sha256:0255b469f0135a0236d672d60e3154ae2f4538b146744966d96440318cc822c6  
Untagged: nginx:latest  
Deleted: sha256:fb197595ebe76b9c0c14ab68159fd3c08bd067ec62300583543f0ebda353b5be
```

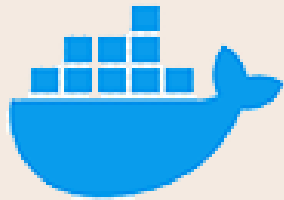
```
# image ls 명령어로 삭제 여부 확인
```

```
docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
------------	-----	----------	---------	------

Docker&Kubernetes

컨테이너 여러 개 연동하기

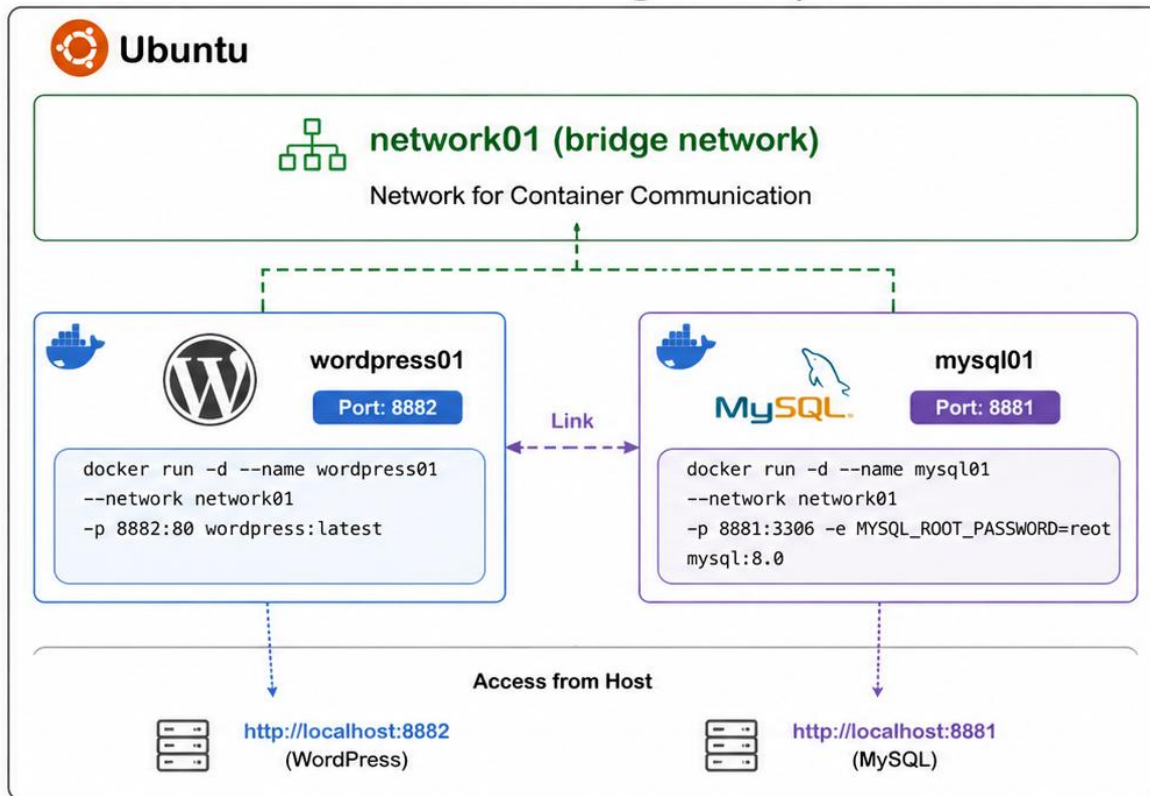


컨테이너 여러 개 연동하기

작업하고자 하는 내용

워드프레스 및 MySql 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

Running Multiple Docker Containers on Ubuntu



Docker Resources

Name	Type	Related Image
network01	Network	Bridge Network
mysql01	Container	mysql:8.0
wordpress01	Container	wordpress:latest

Docker Commands

- # Create Network**
docker network create network01
- # Run MySQL Container**
docker run -d --name mysql01 --network network01 -p 8881:3306 -e MYSQL_ROOT_PASSWORD=root mysql:8.0
- # Run WordPress Container**
docker run -d --name wordpress01 --network network01 -p 8882:80 wordpress:latest



Docker Container



Docker Network

Port: XXXX

Mapped Host Port



Container Command

Network Connection

Container Link

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

워드프레스 및 MySql 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

구현할 내용

가상 네트워크를 생성합니다.
MySql 컨테이너를 작성합니다.
WordPress 컨테이너를 작성합니다.
컨테이너 및 네트워크 정보를 확인합니다.
자원을 정리하고 마무리합니다.

생성할 항목 정보

이름	유형	관련 도커 이미지
network01	네트워크	가상 네트워크
mysql01	컨테이너	mysql
wordpress01	컨테이너	wordpress

구성 내용



WordPress



MySql

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

워드프레스 및 MySql 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

```
# 가상 네트워크 생성하기
# 명령어를 실행하고 나서 생성되는 NETWORK ID는 Network 식별자입니다.
docker network create network01
a4fcceeb6550a72d6ae868ff4bc8b1ed62e46da074e29f8892f732a4572543d7

# 네트워크 목록 확인
docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
5e5db3a7d066	bridge	bridge	local
859302a7bd2d	host	host	local
85247ef88d2e	network01	bridge	local
c8a7b423a58f	none	null	local

👉 개발자가 생성한 가상 네트워크

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

워드프레스 및 MySql 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

```
# 마이 Sql 컨테이너 생성 및 실행
# 최신 MySQL 버전에서는 기본값으로 caching_sha2_password를 사용하므로 mysql_native_password를 명시하지 않아도 됩니다.
docker container run --net=network01 --name=mysql01 -dit -e MYSQL_ROOT_PASSWORD=My@Sql01 -e MYSQL_DATABASE=coffee mysql --
character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
```

```
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
80ba30c57782: Download complete
... 중략
cd0d5df9937b: Download complete
Status: Downloaded newer image for mysql:latest
e5cc6ced6836b1cd9e1d6f8ade0f33bd903147a75007251aac65a1a5af74efea
```

MySql 정보

```
network 이름 : network01
Container 이름 : mysql01
Database 이름 : coffee
관리자 id : root
관리자 비밀번호 : My@Sql01
사용자 id : gomdori
사용자 비밀번호 : Spring@1234
```

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

워드프레스 및 MySql 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

```
# 워드 프레스 컨테이너 생성 및 실행
```

```
docker container run --name=wordpress01 -dit --net=network01 -p=8882:80 -e WORDPRESS_DB_HOST=mysql01 -e  
WORDPRESS_DB_NAME=coffee -e WORDPRESS_DB_USER=root -e WORDPRESS_DB_PASSWORD=My@Sql01 wordpress
```

```
Unable to find image 'wordpress:latest' locally
```

```
latest: Pulling from library/wordpress
```

```
... 중략
```

```
c4f8720ddb1e: Download complete
```

```
4f4fb700ef54: Already exists
```

```
d374174149dd: Download complete
```

```
2586d713206f: Download complete
```

```
73fd6b827991: Download complete
```

```
3b01564181f9: Download complete
```

```
8f2b85a95cfd: Download complete
```

```
Digest: sha256:2f3572d5cd722489fe47d59ed45d947dc9507f5a019eb0567ddf24aedef9257ed
```

```
Status: Downloaded newer image for wordpress:latest
```

```
e6fd01dbdfa8040f57f51f37a14b8915359c723fa93e3a16c0732e0cff3773f2
```

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

워드프레스 및 MySql 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

컨테이너의 상태 확인

STATUS 컬럼의 상태 값이 'Up'이므로 정상적으로 잘 구동이 됨을 알려 줍니다.

```
docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e6fd01dbdfa8	wordpress	"docker-entrypoint.s..."	3 minutes ago	Up 3 minutes	0.0.0.0:8882->80/tcp	wordpress01
e5cc6ced6836	mysql	"docker-entrypoint.s..."	5 minutes ago	Up 5 minutes	3306/tcp, 33060/tcp	mysql01

```
docker network inspect network01
```

컨테이너 영역의 정보만 출력

```
docker network inspect network01 --format '{{json .Containers}}'
```

깔끔하게 Container 이름만 출력

```
docker network inspect network01 --format '{{range .Containers}}{{.Name}}{{"\n"}}{{end}}'
```

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

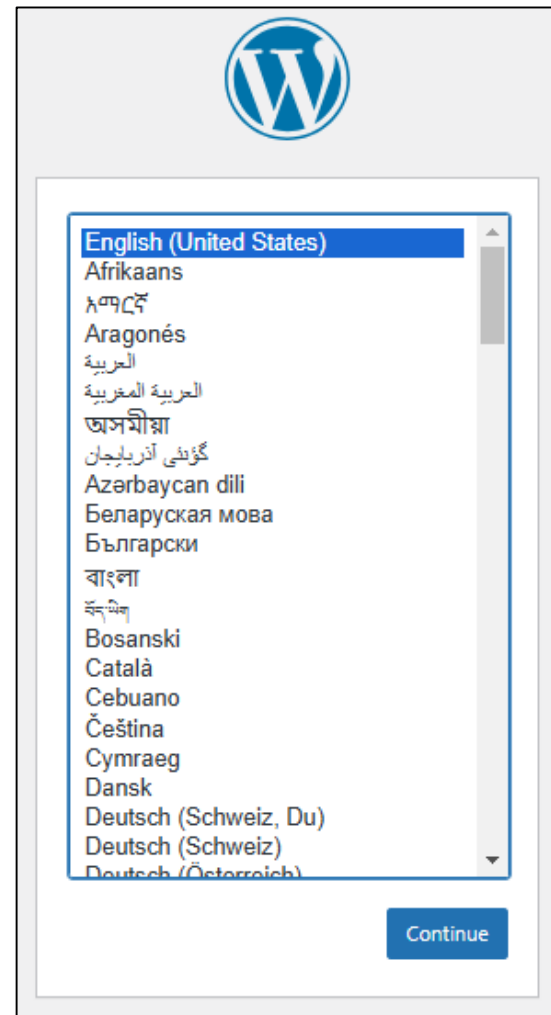
워드프레스 및 MySql 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

브라우저에 다음 주소를 각각 입력하여 화면과 같이 동일하게 출력이 되는지 점검합니다.

포트 번호 8882는 워드 프레스 컨테이너 생성시 기입한 값 입니다.

<http://localhost:8882> (Nat 모드)

<http://xx.xx.xx.xx:8882/> (Bridge 모드)



워드프레스 구동

<http://localhost:8882/>를 입력하여 워드 프레스를 구동합니다.



Magyar
Հայերեն
Bahasa Indonesia
Íslenska
Italiano
日本語
Basa Jawa
ქართული
Taqbaylit
Қазақ тілі
ភាសាខ្មែរ
ಕನ್ನಡ
한국어
کوردی
Кыргызча
ພາສາລາວ
Lietuvių kalba
Latviešu valoda
Македонски јазик
മലയാളം
Монгол

계속

Welcome

Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.

Information needed

Please provide the following information. Do not worry, you can always change these settings later.

Site Title

워드나라

Username

admin

Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.

Password

Admin@123

Hide

Weak

Important: You will need this password to log in. Please store it in a secure location.

Confirm Password

☒ Confirm use of weak password

Your Email

admin@naver.com

Double-check your email address before continuing.

Search engine
visibility

☒ Discourage search engines from indexing this site

It is up to search engines to honor this request.

Install WordPress

≡ 워드프레스 구동

<http://localhost:8882/>를 입력하여 워드 프레스를 구동합니다.

Success!

WordPress has been installed. Thank you, and enjoy!

Username admin

Password *Your chosen password.*

[Log In](#)



사용자명 또는 이메일 주소

admin@naver.com

비밀번호

••••••••

☐ 로그인 상태 저장

로그인

비밀번호를 잊으셨나요?

← 워드나라(으)로 가기



한국어



변경

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

MySQL에서 샘플용 테이블을 생성합니다.

여기서 mysql 명령은 -p 옵션 뒤에 입력된 My@Sql01을 사용자 이름 또는 다른 인자로 인식하려고 해서 오류가 납니다.

MySQL 클라이언트는 -p 뒤에 띄어쓰기 없이 붙여야 비밀번호로 인식합니다.

```
docker exec -it mysql01 mysql -u root -p
```

Enter password: My@Sql01

```
show databases;
```

컨테이너 생성시 Database를 우리가 생성했습니다.

```
USE coffee;
```

```
CREATE TABLE customers(  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50),  
    email VARCHAR(100),  
    regdate DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
INSERT INTO customers(username, email)VALUES ('alice',  
'alice@example.com'),('bob', 'bob@example.com'),('charlie',  
'charlie@example.com');
```

```
commit ;
```

```
exit
```

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

WordPress에서 MySql로 접속하여 데이터를 확인해 봅니다.

```
docker exec -it wordpress01 bash
apt update
apt install -y default-mysql-client
```

```
mysql -h mysql01 -u root -p
비밀번호 입력: My@Sql01
```

```
use coffee;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
```

Database changed

정석은 CA 인증서까지 맞춰서 "정식 SSL 연결"을 해야 합니다.

SSL 비활성화

MySQL 클라이언트

```
mysql -h mysql01 -u root -p --ssl-mode=DISABLED
```

MariaDB 방식으로 접속

```
mysql -h mysql01 -u root -p --skip-ssl
```

```
MySQL [coffee]> select * from my_members;
```

id	username	email	regdate
1	alice	alice@example.com	2026-04-25 09:13:46
2	bob	bob@example.com	2026-04-25 09:13:46
3	charlie	charlie@example.com	2026-04-25 09:13:46

3 rows in set (0.001 sec)

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

워드프레스 및 MySql 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

컨테이너 종료

```
docker container stop mysql01 wordpress01
```

컨테이너 삭제

```
docker container rm mysql01 wordpress01
```

이미지 삭제

```
docker image rm wordpress mysql
```

Untagged: wordpress:latest

Deleted: sha256:2f3572d5cd722489fe47d59ed45d947dc9507f5a019eb0567ddf24aedf9257ed

Untagged: mysql:latest

Deleted: sha256:0255b469f0135a0236d672d60e3154ae2f4538b146744966d96440318cc822c6

네트워크 삭제

```
docker network rm network01
```

```
network01
```

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

레드 마인 및 MariaDB 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

구현할 내용

가상 네트워크를 생성합니다.
MariaDB 컨테이너를 작성합니다.
RedMine 컨테이너를 작성합니다.
컨테이너 및 네트워크 정보를 확인합니다.
자원을 정리하고 마무리합니다.

생성할 항목 정보

이름	유형	관련 도커 이미지
network02	네트워크	가상 네트워크
mariadb02	컨테이너	mariadb
redmine02	컨테이너	redmine

구성 내용



Redmine



MariaDB

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

레드 마인 및 MariaDB 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

가상 네트워크 생성하기

명령어를 실행하고 나서 생성되는 NETWORK ID는 Network 식별자입니다.

```
docker network create network02
```

네트워크 목록 확인

```
docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
5e5db3a7d066	bridge	bridge	local
859302a7bd2d	host	host	local
c3fc8bf39618	network02	bridge	local
c8a7b423a58f	none	null	local

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

레드 마인 및 MariaDB 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

```
# mariadb 컨테이너 생성 및 실행하기
docker run --net=network02 --name=mariadb02 -dit -e MYSQL_ROOT_PASSWORD=My@Sql01 -e MYSQL_DATABASE=coffee mariadb --character-
set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
Unable to find image 'mariadb:latest' locally
latest: Pulling from library/mariadb
407e9d6eefb4: Download complete
de44b265507a: Download complete
ca9b21d0c985: Download complete
041c753879ae: Download complete
e7b5137dc4b2: Download complete
655e5d2590bd: Download complete
95adc28016bc: Download complete
41b3170b5f12: Download complete
Digest: sha256:a9547599cd87d7242435aea6fda22a9d83e2c06d16c658ef70d2868b3d3f6a80
Status: Downloaded newer image for mariadb:latest
f4d3871e4ed445ddfbfd586b5e83aaca4124466cab8294388abbdfa98bcccb09c
```

≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

레드 마인 및 MariaDB 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

```
# redmine 컨테이너 생성 및 실행하기
```

```
docker run --net=network02 --name=redmine02 -dit -p=8883:3000 -e REDMINE_DB_MYSQL=mariadb02 -e REDMINE_DB_DATABASE=coffee -e REDMINE_DB_USERNAME=root -e REDMINE_DB_PASSWORD=My@Sql01 redmine
```

```
Unable to find image 'redmine:latest' locally
```

```
latest: Pulling from library/redmine
```

```
59c0e5bc9df9: Download complete
```

```
... 중략
```

```
1fcde8448b1: Download complete
```

```
72e843b9fb77: Download complete
```

```
59a3b7d1323e: Download complete
```

```
Digest: sha256:c0740f9579042646fd137a267b74fc28d7ec9899479cf23433a6c357d928755b
```

```
Status: Downloaded newer image for redmine:latest
```

```
08394b8c811165deabcb8b41449fe285d5f777143275f0186ab5a9ead6e8a147
```

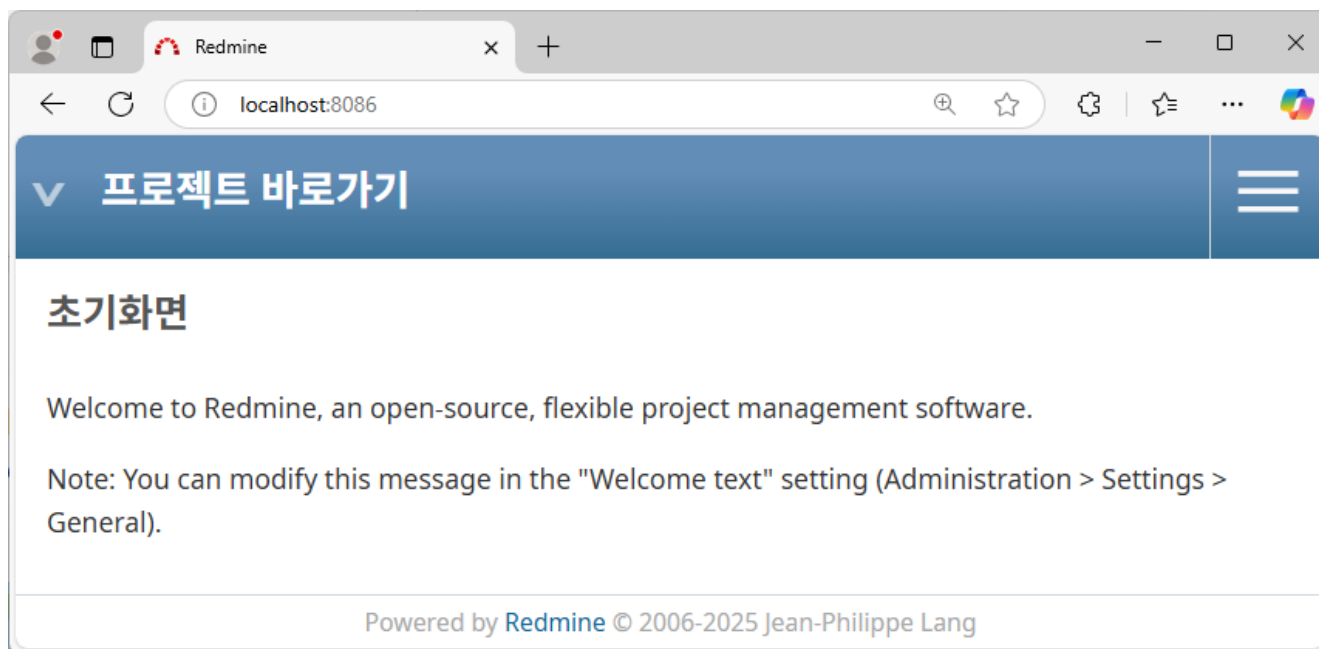
≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

레드 마인 및 MariaDB 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

브라우저에 다음 주소를 각각 입력하여 화면과 같이 동일하게 출력이 되는지 점검합니다

<http://localhost:8883>



≡ 컨테이너 여러 개 연동하기

≡ 작업하고자 하는 내용

레드 마인 및 MariaDB 컨테이너 생성과 연동 작업을 수행해 보겠습니다.

```
docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
4e6adc76bad8	redmine	"/docker-entrypoint...."	13 minutes ago	Up 12 minutes ago	redmine02	
f4d3871e4ed4	mariadb	"docker-entrypoint.s..."	30 minutes ago	Up 30 minutes	3306/tcp	mariadb02

```
docker container stop mariadb02
```

```
docker container rm mariadb02
```

```
docker container stop redmine02
```

```
docker container rm redmine02
```

```
docker image rm mariadb
```

```
Untagged: mariadb:latest
```

```
Deleted: sha256:a9547599cd87d7242435aea6fda22a9d83e2c06d16c658ef70d2868b3d3f6a80
```

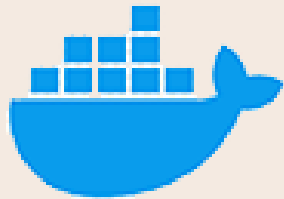
```
docker image rm redmine
```

```
Untagged: redmine:latest
```

```
Deleted: sha256:c0740f9579042646fd137a267b74fc28d7ec9899479cf23433a6c357d928755b
```

Docker&Kubernetes

실전 활용 컨테이너 사용법



≡ 실전 활용 컨테이너 사용법

≡ 사전 준비

dk04.docker_실습_파일.zip 파일을 압축 해제 하여 파일을 미리 준비합니다.

Window

D:\docker\ 폴더에 압축 해제하기

Linux

~/docker 디렉토리에 압축 해제하기

```
cd
```

```
mkdir docker
```

```
sudo apt install -y zip
```

```
# 하위 디렉토리 docker에 압축 해제하세요.
```

```
unzip docker_test.zip -d docker
```

```
cd docker
```

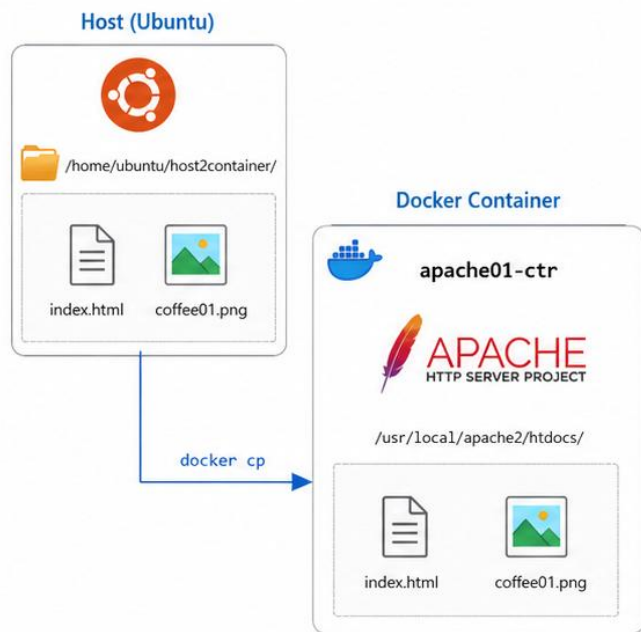
```
ls
```

실전 활용 컨테이너 사용법

파일 복사(from Host to Container)

Host 미리 작성해 놓은 파일을 Container으로 복사하는 테스트를 진행해 보겠습니다.

Copy Files from Host (Ubuntu) to Docker Container (Apache)



1 Start Apache Container (httpd)

```
docker container run --name apache01-ctr -d -p 8090:80 httpd
```

✓ Container started. Apache is running on port 8090.

2 Check files in Apache document root

```
docker exec apache01-ctr ls /usr/local/apache2/htdocs
```

```
# (initially empty or default files)
index.html # default file from httpd image
```

3 View default index.html content

```
docker exec apache01-ctr cat /usr/local/apache2/htdocs/index.html
```

```
<html><body><h1>It works!</h1>
<p>This is the default web page for this server.</p>
</body></html>
```

4 Copy files from Host to Container

```
docker cp d:\docker\host2container\index.html apache01-ctr:/usr/local/apache2/htdocs
```

✓ index.html copied.

```
docker cp d:\docker\host2container\coffee01.png apache01-ctr:/usr/local/apache2/htdocs
```

✓ coffee01.png copied.

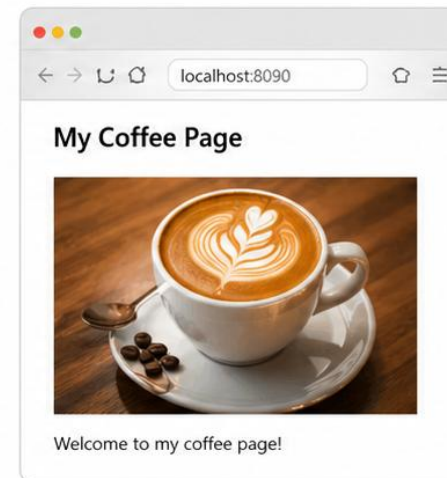
5 Verify files in container

```
docker exec apache01-ctr ls /usr/local/apache2/htdocs
```

```
index.html
coffee01.png
```

6 Access in Browser

<http://localhost:8090>



실전 활용 컨테이너 사용법

파일 복사(from Host to Container)

Host 미리 작성해 놓은 파일을 Container으로 복사하는 테스트를 진행해 보겠습니다.

도커내 파일 복사 실습(apache).txt

```
# 호스트 컴퓨터의 파일을 컨테이너에 복사하기  
# 드라이브 d:\docker\host2container\ 경로에 다음과 같은 파일을 미리 작성해 둡니다.  
# index.html, coffee01.png
```

index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Title</title>  
</head>  
<body>  
  <h3>Coffee Shop</h3>  
  따뜻한 <span style="color:blue;">아메리카노</span>  
    <br/><br/><br/>  
    
</body>  
</html>
```

coffee01.png



≡ 실전 활용 컨테이너 사용법

≡ 파일 복사(from Host to Container)

Host 미리 작성해 놓은 파일을 Container으로 복사하는 테스트를 진행해 보겠습니다.

아파치 컨테이너 생성

```
docker container run --name apache01-ctr -d -p 8090:80 httpd
```

브라우저에서 구동되는 지 확인

<http://localhost:8090>

웹 브라우저에 다음과 같이 출력이 되어야 합니다.

It works!

≡ 실전 활용 컨테이너 사용법

≡ 파일 복사(from Host to Container)

Window 실습) Host 미리 작성해 놓은 파일을 Container으로 복사하는 테스트를 진행해 보겠습니다.

특정 디렉토리 내의 파일 목록 확인

```
docker exec apache01-ctr ls /usr/local/apache2/htdocs
```

index.html 파일 내용 확인

```
docker exec apache01-ctr cat /usr/local/apache2/htdocs/index.html
```

cp 명령어를 이용하여 `호스트 컴퓨터`에서 `컨테이너`로 파일 복사하기

여기서 /usr/local/apache2/htdocs는 Apache HTTP 서버의 기본 웹 루트 디렉토리

```
docker cp d:\docker\host2container\index.html apache01-ctr:/usr/local/apache2/htdocs
```

Successfully copied 2.05kB to apache01-ctr:/usr/local/apache2/htdocs

```
docker cp d:\docker\host2container\coffee01.png apache01-ctr:/usr/local/apache2/htdocs
```

Successfully copied 197kB to apache01-ctr:/usr/local/apache2/htdocs

브라우저에서 다시 구동해 봅니다.

내가 작성한 문서가 홈 페이지로 보여야 합니다.(결과 그림 참조)

```
# http://localhost:8090
```



Coffee Shop

따뜻한 아메리카노



≡ 실전 활용 컨테이너 사용법

≡ 파일 복사(from Container to Host)

Window 실습) Container 내에 존재하는 파일들을 Host 컴퓨터로 복사하는 테스트를 진행해 보겠습니다.

```
# 윈도우에 d:\docker\ 폴더 미리 생성해두기
# cp 명령어를 이용하여 `호스트`에서 `컨테이너`로 파일 복사하기
# index.html 문서를 index2.html 문서로 복사합니다.
docker cp apache01-ctr:/usr/local/apache2/htdocs/index.html d:\docker\hello.html
Successfully copied 2.05kB to d:\docker\index2.html

docker cp apache01-ctr:/usr/local/apache2/htdocs/coffee01.png d:\docker\world.png
# coffee01.jpg 문서를 coffee02.jpg 이미지로 복사합니다.
Successfully copied 197kB to d:\docker\coffee02.png
```


≡ 실전 활용 컨테이너 사용법

≡ 파일 복사(from Container to Host)

Window 실습) Container 내에 존재하는 파일들을 Host 컴퓨터로 복사하는 테스트를 진행해 보겠습니다.

d:\docker\ 폴더를 확인해 봅니다.

`dir d:\docker`

D 드라이브의 볼륨: 새 볼륨

볼륨 일련 번호: 2065-AC31

d:\docker 디렉터리

2024-12-10	오후 03:46	<DIR>	.
2024-12-10	오후 03:46	<DIR>	..
2023-05-09	오후 04:00	194,622	world.png
2024-11-23	오후 06:26	294	hello.html
	2개 파일	194,916	바이트
	2개 디렉터리	995,351,478,272	바이트 남음

≡ 실전 활용 컨테이너 사용법

≡ 파일 복사(from Host to Container)

Linux 실습) Host 미리 작성해 놓은 파일을 Container으로 복사하는 테스트를 진행해 보겠습니다.

특정 디렉토리 내의 파일 목록 확인

```
docker exec apache01-ctr ls /usr/local/apache2/htdocs
```

index.html 파일 내용 확인

```
docker exec apache01-ctr cat /usr/local/apache2/htdocs/index.html
```

from Host to Container

cp 명령어를 이용하여 `호스트 컴퓨터`에서 `컨테이너`로 파일 복사하기

여기서 /usr/local/apache2/htdocs는 Apache HTTP 서버의 기본 웹 루트 디렉토리

```
docker cp docker/host2container/index.html apache01-ctr:/usr/local/apache2/htdocs/index.html
```

```
docker cp docker/host2container/coffee01.png apache01-ctr:/usr/local/apache2/htdocs/coffee01.png
```

브라우저에서 다시 구동해 봅니다.

내가 작성한 문서가 홈 페이지로 보여야 합니다.(결과 그림 참조)

<http://localhost:8090>



Coffee Shop

따뜻한 아메리카노



≡ 실전 활용 컨테이너 사용법

≡ 파일 복사(from Container to Host)

Linux 실습) Container 내에 존재하는 파일들을 Host 컴퓨터로 복사하는 테스트를 진행해 보겠습니다.

```
# from Container to Host
```

```
docker cp apache01-ctr:/usr/local/apache2/htdocs/index.html docker/host2container/hello.html
```

```
docker cp apache01-ctr:/usr/local/apache2/htdocs/coffee01.png docker/host2container/world.png
```

☰ 실전 활용 컨테이너 사용법

☰ 파일 복사(from Container to Host)

Linux 실습) Container 내에 존재하는 파일들을 Host 컴퓨터로 복사하는 테스트를 진행해 보겠습니다.

```
ls docker/host2container/
```

```
coffee01.png  hello.html  index.html  world.png
```

≡ 실전 활용 컨테이너 사용법

≡ 파일 복사(from Container to Host)

Container 내에 존재하는 파일들을 Host 컴퓨터로 복사하는 테스트를 진행해 보겠습니다.

컨테이너 멈춤

```
docker container stop apache01-ctr
```

```
apache01-ctr
```

컨테이너 삭제

```
docker container rm apache01-ctr
```

```
apache01-ctr
```

실전 활용 컨테이너 사용법

실습해 보기

풀어 봅시다.

다음 요구 사항대로 컨테이너 생성 및 실행, 상태 확인, 종료 등을 실습해 보도록 합니다.

도커내 파일 복사 실습(nginx).txt

요구 사항

```
# 컨테이너 이름 : nginx88
# 이미지 이름 : nginx
# 포트 번호 : 8888
# 내가 만든 홈페이지 파일을 복사
# 반영이 잘 되는 체크
# 컨테이너 종료 후 삭제
# index.html 위치 : /usr/share/nginx/html/
```

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Apache 웹 서버가 정상적으로 동작합니다!

브라우저에서 웹 서버 테스트 페이지입니다.



≡ 실전 활용 컨테이너 사용법

≡ 저장소(바인드 마운트) 실습

바인드 마운트는 운영 체제의 일부 공간(디렉토리)을 컨테이너에 마운트 하는 방식입니다.

```
# 윈도우 d:\bind_mount 폴더를 미리 생성해 두고, 이 폴더를 마운트 원본이 될 폴더로 지정하겠습니다.
# 참고로 이 폴더를 미리 생성하지 않더라도 자동으로 생성해 줍니다.
# Window) 아파치 컨테이너 생성 및 실행
docker container run --name apache02-ctr -d -p 8090:80 -v D:\bind_mount:/usr/local/apache2/htdocs httpd

# Linux) 홈 디렉토리 아래의 bind_mount와 아파치 홈 디렉토리와 바인딩시켜 주세요.
# docker container run --name apache02-ctr -d -p 8090:80 -v ~/bind_mount:/usr/local/apache2/htdocs httpd

# d:\bind_mount 폴더 생성이 되었는 지 확인
ls
cd bind_mount/
ls -al

# 브라우저에서 구동되는 지 확인
# http://localhost:8090를 실행하면 `Index of /`라는 결과를 출력합니다.
```

Index of /

≡ 실전 활용 컨테이너 사용법

≡ 저장소(바인드 마운트) 실습

바인드 마운트는 운영 체제의 일부 공간(디렉토리)을 컨테이너에 마운트 하는 방식입니다.

inspect 명령어를 사용하면 JSON 형식으로 결과가 반환됩니다.

컨테이너의 설정, 상태, 네트워크, 볼륨, 마운트 정보 등 모든 내부 메타데이터를 확인할 수 있습니다.

`Mounts` 항목을 살펴 보면 됩니다.

```
docker container inspect apache02-ctr
```

```
"Mounts": [  
  {  
    "Type": "bind",  
    "Source": "D:\\bind_mount",  
    "Destination": "/usr/local/apache2/htdocs",  
    "Mode": "",  
    "RW": true,  
    "Propagation": "rprivate"  
  }  
]
```

특정 글자가 들어 있는 행만 보기

```
docker container inspect apache02-ctr | grep -E "Source|Destination|bind"
```

항목	설명
Source	호스트(내 컴퓨터)의 디렉토리 경로 즉, Docker 바깥쪽(Windows)에서 실제 파일이 있는 위치를 의미
Destination	컨테이너 내부의 경로를 말합니다. 즉, 컨테이너 안의 Apache 웹 서버 DocumentRoot 위치를 의미합니다.

≡ 실전 활용 컨테이너 사용법

≡ 저장소(바인드 마운트) 실습

바인드 마운트는 운영 체제의 일부 공간(디렉토리)을 컨테이너에 마운트 하는 방식입니다.

파일 복사

```
# 내 컴퓨터에서 마운트 폴더(bind_mount 폴더)에 다음 파일을 복사하도록 합니다.  
# 첨부 파일 index_bind.html를 복사하여 index.html로 이름 변경, image19_02.png  
sudo cp ~/docker/bind_mount/* ~/bind_mount/  
cd ~/bind_mount/  
sudo mv index_bind.html index.html  
  
# 브라우저에서 실행 결과가 다르게 보이는 지 확인합니다.  
# http://localhost:8090 F5 키를 눌러서 화면이 변경되는지 확인합니다.
```

D:\bind_mount 안에 index.html 파일을 복사해 두면
컨테이너의 /usr/local/apache2/htdocs에 그대로 반영이 됩니다.

[Windows 디렉토리]
D:\bind_mount\index.html

↓ (bind mount)

[컨테이너 내부]
/usr/local/apache2/htdocs/index.html

벚꽃 놀이

따뜻한 봄이 왔네요^^



≡ 실전 활용 컨테이너 사용법

≡ 저장소(바인드 마운트) 실습

바인드 마운트는 운영 체제의 일부 공간(디렉토리)을 컨테이너에 마운트 하는 방식입니다.

```
# 아파치 컨테이너 중지  
docker container stop apache02-ctr  
apache02-ctr  
  
# 아파치 컨테이너 삭제  
docker container rm apache02-ctr  
apache02-ctr
```

실전 활용 컨테이너 사용법

저장소(바인드 마운트) 실습

풀어 봅시다.

다음 요구 사항대로 컨테이너 생성 및 실행, 상태 확인, 종료 등을 실습해 보도록 합니다.

바인드 마운트 실습

Container 이름 : nginx30-ctr

Image 이름 : nginx

포트 번호 : 8830

디렉토리 : ~/bind_nginx

실습에 사용할 파일 :

~/docker/index03.html

~/docker/image10_02.png

이미지 기본 root

httpd /usr/local/apache2/htdocs/index.html

nginx /usr/share/nginx/html/index.html

```
docker container run --name nginx30-ctr -d -p 8830:80 -v
~/bind_nginx:/usr/share/nginx/html nginx
cd bind_nginx/
```

http://192.168.0.34:8830 ==> 403 Forbidden 발생

nginx는 아파치와 다르게 bind_nginx 디렉토리에 파일을 먼저 넣어 주어야 합니다.

```
docker container inspect nginx30-ctr | grep -E "SourceDestination"
```

```
sudo cp ~/docker/index03.html ~/bind_nginx/index.html
```

Apache 웹 서버가 정상적으로 동작합니다!

브라우저에서 웹 서버 테스트 페이지입니다.

```
sudo cp ~/docker/image10_02.png ~/bind_nginx/
```

```
# http://192.168.0.34:8830
```



≡ 실전 활용 컨테이너 사용법

≡ 저장소(볼륨 마운트) 실습

도커 엔진이 관리하는 영역 내에 만들어진 볼륨을 컨테이너에 디스크 형태로 마운트합니다.

마운트할 볼륨을 생성합니다.

```
docker volume create mount-vol
```

```
mount-vol
```

아파치 컨테이너 생성 및 실행

```
docker container run --name apache03-ctr -d -p 8090:80 -v mount-vol:/usr/local/apache2/htdocs httpd  
9bd8a28aa53fe7eaac9994c9c5e353594a98cd338a404bc95737db966682fef3
```

≡ 실전 활용 컨테이너 사용법

≡ 저장소(볼륨 마운트) 실습

도커 엔진이 관리하는 영역 내에 만들어진 볼륨을 컨테이너에 디스크 형태로 마운트합니다.

```
# 볼륨의 상세 정보 확인
# Mountpoint에 명시된 디렉토리는 도커가 관리하고 있으므로, 직접 변경은 비권장입니다.
# 굳이 변경을 하려고 한다면 컨테이너를 통해 조작하기를 권장합니다.
```

```
docker volume inspect mount-vol
```

```
[
  {
    "CreatedAt": "2024-12-10T07:37:54Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mount-vol/_data",
    "Name": "mount-vol",
    "Options": null,
    "Scope": "local"
  }
]
```

항목	설명
Mountpoint	호스트(리눅스 또는 WSL2)의 실제 디스크에 있는 디렉터리 위치입니다.

```
docker volume inspect mount-vol | grep -E "Mountpoint|Name"
```

≡ 실전 활용 컨테이너 사용법

≡ 저장소(볼륨 마운트) 실습

도커 엔진이 관리하는 영역 내에 만들어진 볼륨을 컨테이너에 디스크 형태로 마운트합니다.

```
# 볼륨이 컨테이너에 잘 마운트 되었는 지 확인
docker container inspect apache03-ctr > d:\cc.txt

# d:\cc.txt 파일에서 `Mounts`라는 항목을 찾아 보세요.
d:\cc.txt
"Mounts": [
    {
        "Type": "volume",
        "Name": "mount-vol",
        "Source": "/var/lib/docker/volumes/mount-vol/_data",
        "Destination": "/usr/local/apache2/htdocs",
        "Driver": "local",
        "Mode": "z",
        "RW": true,
        "Propagation": ""
    }
]
```

≡ 실전 활용 컨테이너 사용법

≡ 저장소(볼륨 마운트) 실습

도커 엔진이 관리하는 영역 내에 만들어진 볼륨을 컨테이너에 디스크 형태로 마운트합니다.

아파치 컨테이너 중지

```
docker container stop apache03-ctr
```

```
apache03-ctr
```

아파치 컨테이너 삭제

```
docker container rm apache03-ctr
```

```
apache03-ctr
```

이미지 삭제

```
docker image rm httpd
```

```
Untagged: httpd:latest
```

```
Deleted: sha256:f4c5139eda466e45814122d9bd8b886d8ef6877296126c09b76dbad72b03c336
```

볼륨 삭제

```
docker volume rm mount-vol
```

```
mount-vol
```

≡ 실전 활용 컨테이너 사용법

≡ 컨테이너로 이미지 만들기

컨테이너를 수정하고, commit 명령어를 이용하여 새로운 이미지를 만들어 봅니다.

생성 방법

항목	설명
방법01	commit 명령어를 이용하여 이미지를 만들어 봅니다.
방법02	Dockerfile 스크립트를 이용하여 이미지를 만들어 봅니다.

≡ 실전 활용 컨테이너 사용법

≡ 이미지 생성(commit)

컨테이너를 수정하고, commit 명령어를 이용하여 새로운 이미지를 만들어 봅니다.

```
# 방법01 : commit 명령어를 이용하여 이미지를 만들어 봅니다.
docker container run --name commit-ctr -d -p 8090:80 httpd

# 파일 복사(Window)
docker cp d:\docker\commit\index04.html commit-ctr:/usr/local/apache2/htdocs/index.html
docker cp d:\docker\commit\image17_02.png commit-ctr:/usr/local/apache2/htdocs

# 파일 복사(Linux)
docker cp ~/docker/commit/index04.html commit-ctr:/usr/local/apache2/htdocs/index.html
docker cp ~/docker/commit/image17_02.png commit-ctr:/usr/local/apache2/htdocs/image17_02.png

# `exec`은 실행 중인 컨테이너 안에서 명령어를 실행하도록 할게요.
# `-it`는 터미널에서 키보드 입력이 가능하도록 할게요.
# 터미널로 진입하여 `commit-ctr` 컨테이너 내부로 접속하여 파일을 작성합니다.
docker exec -it commit-ctr /bin/bash
root@0bdd59c957be:/usr/local/apache2#

# 컨테이너 내부에서 텍스트 파일을 `/usr/local/apache2/htdocs` 디렉터리에 작성합니다.
echo "This is jeju test" > /usr/local/apache2/htdocs/jeju.txt
```

귤의 정원 바령

제주도~~2022년도



≡ 실전 활용 컨테이너 사용법

≡ 이미지 생성(commit)

컨테이너를 수정하고, commit 명령어를 이용하여 새로운 이미지를 만들어 봅니다.

파일 작성이 잘 되었는 지 cat 명령어를 이용하여 확인합니다.

```
root@0bdd59c957be:/usr/local/apache2# cat /usr/local/apache2/htdocs/jeju.txt
```

This is jeju test

파일 작성이 완료되었으므로 컨테이너에서 빠져 나옵니다.

```
root@0bdd59c957be:/usr/local/apache2# exit
```

컨테이너를 사용하여 새로운 이미지를 생성합니다.

```
docker commit commit-ctr jeju-img
```

```
sha256:e30f47b0f3e1e0f0832f73a4913dd6bacaa5805942ad7dbada587bbad6d343b5
```

Images 열에 해당 이미지 이름이 생성이 되어 있는 지 확인하도록 합니다.

```
docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
jeju-img:latest	5771d6cc0d0a	182MB	48.9MB ← 이미지 정보가 보임	
httpd:latest	8f5166aa2f6d	175MB	45.2MB	U

≡ 실전 활용 컨테이너 사용법

≡ 이미지 생성(commit)

컨테이너를 수정하고, commit 명령어를 이용하여 새로운 이미지를 만들어 봅니다.

```
# jeju-img라는 도커 이미지를 이용하여 jeju-ctr라는 컨테이너 생성하기.  
docker container run --name jeju-ctr -d -p 8100:80 jeju-img  
649431296bc721a9a819579a655ec9758c4145bf75557a1d86238bbdd2856960  
  
# 신규 컨테이너에 jeju.txt라는 파일이 존재하는 지 확인하도록 합니다.  
docker exec jeju-ctr cat /usr/local/apache2/htdocs/jeju.txt  
This is jeju test  
  
# 브라우저에서 확인  
http://localhost:8100
```

귤의 정원 바령

제주도~~2022년도



≡ 실전 활용 컨테이너 사용법

≡ 이미지 생성(commit)

관련 항목들을 모두 종료시키고, 삭제하도록 합니다.

해당 컨테이너와 이미지 정보들을 종료하고, 삭제하도록 합니다.

```
docker container stop jeju-ctr
```

```
docker container rm jeju-ctr
```

```
docker image rm jeju-img
```

이미지 생성(commit)

[풀어 봅시다.](#)

Container와 Image 생성 및 실행, 상태 확인, 종료 등에 대한 실습해 보도록 합니다.

이미지 생성 실습(commit_nginx).txt

실습 내용 01

Step01 표를 보고, 다음과 같이 실습해 주세요.

다음 파일로 홈페이지를 교체하세요.

~/docker/dockerfile/index05.html

~/docker/dockerfile/image18_02.png

echo와 리다이렉션으로 nginx.txt 텍스트 파일을 하나 생성해 보세요.
브라우저 테스트를 진행하세요.

실습 내용 02

Step02 표를 보고, 다음과 같이 실습해 주세요.

사용자 정의 image를 사용하여 컨테이너를 생성해 보세요.

nginx.txt 텍스트 파일을 내용이 잘 보이는 지 확인해 보세요.

브라우저 테스트를 진행하세요.

Step01

항목	설명
docker image	nginx
생성할 컨테이너	nginx-commit-ctr
Port no	8880
신규 생성 이미지	pohang-nginx-img

Step02

항목	설명
사용자 정의 image	pohang-nginx-img
생성할 컨테이너	pohang-nginx-ctr
Port no	8400

≡ 실전 활용 컨테이너 사용법

≡ 이미지 생성(Dockerfile)

Dockerfile 스크립트를 이용하여 이미지를 만들어 봅니다.

```
# 방법02 : Dockerfile 스크립트를 이용하여 이미지를 만들어 봅니다.  
# 메모장을 사용하여 Dockerfile01 파일을 작성하도록 합니다.  
# d:\docker\dockerfile\ 폴더 내에 다음과 같은 파일들이 존재하는 지 확인합니다.  
#      1. Dockerfile01, 2. index05.html, 3. image18_02.png
```

Dockerfile 스크립트

`from httpd` 명령어는 Docker Hub에서 httpd라는 이름의 이미지(=Apache HTTP Server)를 다운로드해서 사용하겠다는 의미입니다.

"copy index05.html /usr/local/apache2/htdocs/index.html" 명령어는 호스트의 index05.html 파일을 컨테이너 내부의 웹 루트 디렉토리에 index.html 이라는 파일 이름으로 복사하겠습니다.

```
from httpd  
copy index05.html /usr/local/apache2/htdocs/index.html  
copy image18_02.png /usr/local/apache2/htdocs
```

≡ 실전 활용 컨테이너 사용법

≡ 컨테이너로 이미지 만들기

Dockerfile 스크립트를 이용하여 이미지를 만들어 봅니다.

build 커맨드로 d:\docker\dockerfile의 내용을 pohang-img라는 이미지로 빌드합니다.

윈도우

```
docker build -t pohang-img -f d:\docker\dockerfile\Dockerfile01 d:\docker\dockerfile
```

리눅스

```
docker build -t pohang-img -f ~/docker/dockerfile/Dockerfile01 .
```

[+] Building 2.3s (7/7) FINISHED

docker:desktop-linux

=> [internal] load build definition from Dockerfile

0.0s

=> [internal] load build context

... 종락

=> exporting manifest sha256:ef550441841bbb5a7a6c1acac5a702643473d6598f5073875c5741082df4d326

0.0s

=> unpacking to docker.io/library/pohang-img:latest

실전 활용 컨테이너 사용법

컨테이너로 이미지 만들기

Dockerfile 스크립트를 이용하여 이미지를 만들어 봅니다.

pohang-img라는 이미지가 생성이 되어 있는 지 확인합니다.

`docker image ls`

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
httpd:latest	8f5166aa2f6d	175MB	45.2MB	U
pohang-img:latest	1d01716b035a	180MB	47.9MB	

컨테이너 생성 여부 확인 : pohang-img 이미지를 사용하여 생성된 컨테이너가 존재하지 않습니다.

`docker container ps -a`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
649431296bc7	jeju_apache	"httpd-foreground"	6 minutes ago	Up 6 minutes	80/tcp	jeju_container
0bdd59c957be	httpd	"httpd-foreground"	52 minutes ago	Up 52 minutes	0.0.0.0:8090->80/tcp	apache01

pohang-img를 사용하여 pohang-ctr를 생성합니다.

`docker container run --name pohang-ctr -d -p 8989:80 pohang-img`

≡ 실전 활용 컨테이너 사용법

≡ 컨테이너로 이미지 만들기

브라우저에서 확인하고, 관련 항목들을 모두 종료시키고, 삭제하도록 합니다.

IP 확인

```
ip a show enp0s3
```

브라우저 확인

```
http://xx.xx.xx.xx:8989/
```

컨테이너와 새롭게 생성한 이미지들을 삭제합니다.

```
docker container stop commit-ctr
```

```
docker container rm commit-ctr
```

```
docker container stop pohang-ctr
```

```
docker container rm pohang-ctr
```

```
docker image rm pohang-img
```

포항 호미곶

2024년도 상생의 손 방문



이미지 생성(Dockerfile)

[풀어 보시다.](#)

Container와 Image 생성 및 실행, 상태 확인, 종료 등에 대한 실습해 보도록 합니다.

이미지 생성 실습(dockerfile_nginx).txt

실습 내용 01

Step01 표를 보고, 다음과 같이 실습해 주세요.

다음 파일을 사용하여 Dockerfile을 생성해 주세요.

~/docker/commit/image17_02.png

~/docker/commit/index04.html

미리 만들어 놓은 docker.txt 텍스트 파일을 복사해야 합니다.

docker build 명령어를 사용하여 사용자 정의 Image를 생성해 주세요.

Image 생성이 잘 되었는지 확인해 보세요.

사용자 정의 Image를 사용하여 신규 컨테이너를 생성해 보세요.

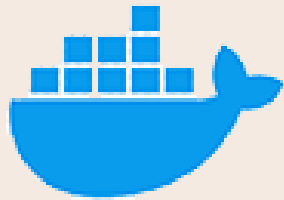
브라우저 테스트를 진행하세요.

Step01

항목	설명
docker image	nginx
생성할 Dockerfile	Dockerfile02
신규 생성 이미지	pohang-dockerfile-img
신규 컨테이너	pohang-dockerfile-ctr
Port no	8500

Docker&Kubernetes

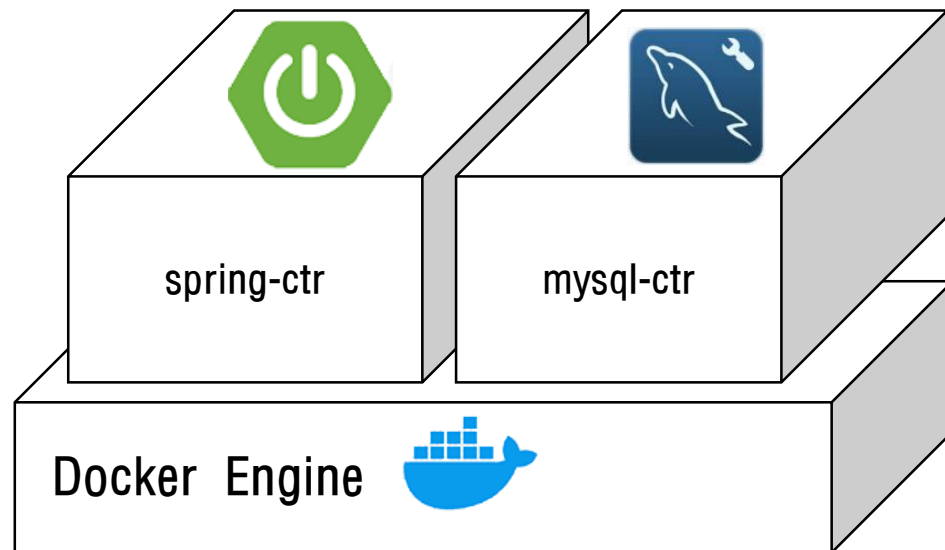
DockerFile 응용



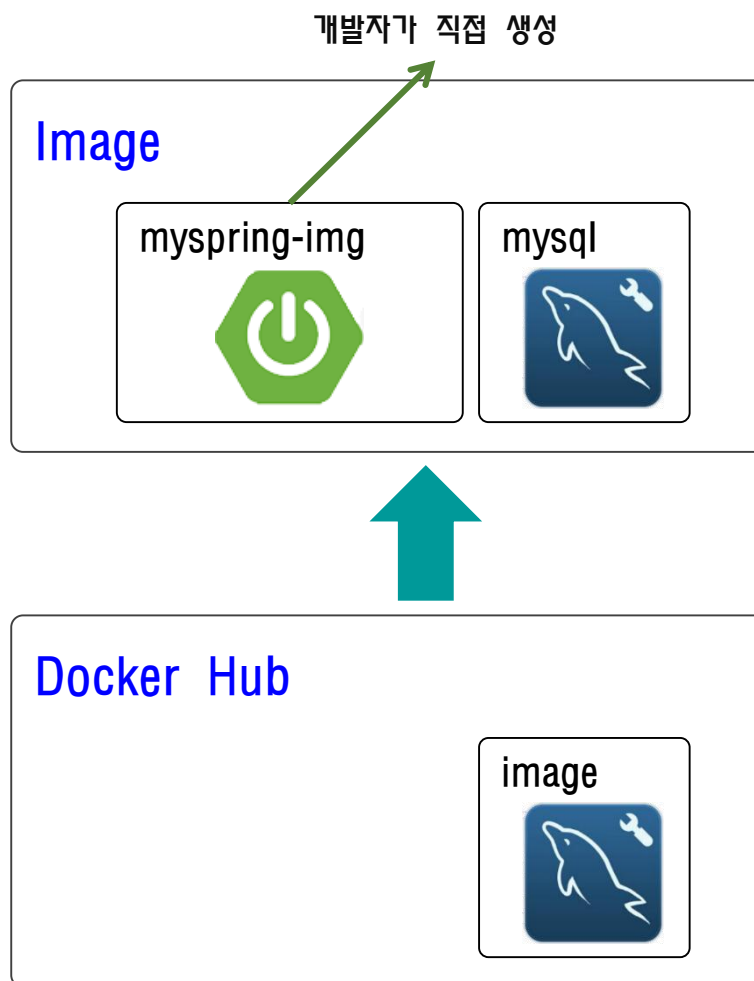
≡ DockerFile 응용

≡ SpringBoot와 Mysql 연동

사용자 정의 SpringBoot 이미지와 Mysql 이미지를 사용하여 간단한 웹 서버를 구현해 봅니다.



← 이미지로 컨테이너
생성하기



≡ SpringBoot와 Mysql 연동

사용자 정의 SpringBoot 이미지와 Mysql 이미지를 사용하여 간단한 웹 서버를 구현해 봅니다.

구현할 내용

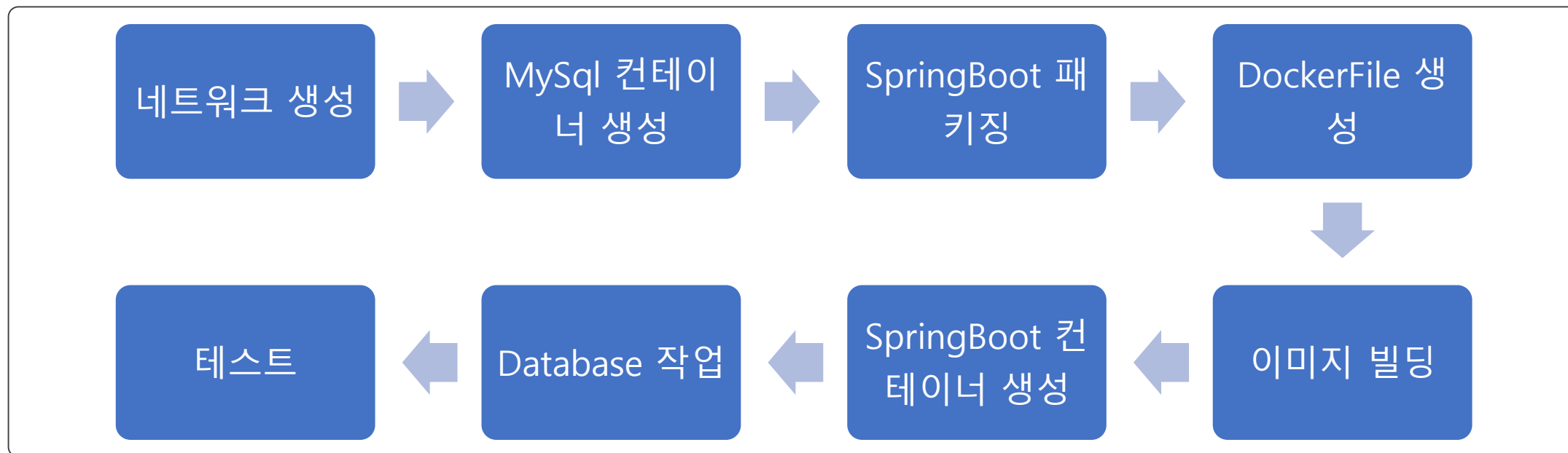
명명 내용	이름
네트워크 이름	spring-mysql-net
Mysql 컨테이너 이름	mysql-ctr
사용할 데이터 베이스 이름	coffee
사용자 정의 이미지 이름	myspring-img
Docklerfile 이름	Docklerfile01
스프링 컨테이너 이름	spring-ctr

≡ DockerFile 응용

≡ SpringBoot와 Mysql 연동

SpringBoot와 Mysql 연동 테스트를 실습 순서는 다음과 같습니다.

실습 순서



≡ Docker 네트워크 생성

SpringBoot와 Mysql를 연동하기 위한 네트워크를 우선 생성합니다.

네트워크 생성

`docker network create spring-mysql-net`

50781e617df0ca7f33791230eecba8574b48ff86de95d2d0b89ee0f720a927bc

Docker 네트워크 목록 전체 확인

`docker network ls`

NETWORK ID	NAME	DRIVER	SCOPE
95468567f2b9	bridge	bridge	local
cceafb7b9408	host	host	local
83127525feda	none	null	local
0ac5caf9a4b0	spring-mysql-net	bridge	local

≡ MySQL Docker 컨테이너 생성

Mysql 이미지를 사용하여 컨테이너를 생성하고 실행합니다.

Mysql 컨테이너 생성

```
docker container run --net=spring-mysql-net --name=mysql-ctr -dit -e MYSQL_ROOT_PASSWORD=My@Sql01 -e MYSQL_DATABASE=coffee -e MYSQL_USER=gomdori -e MYSQL_PASSWORD=Spring@1234 mysql --character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
```

옵션 항목

항목	설명
MYSQL_ROOT_PASSWORD	root 사용자 계정 비밀번호
MYSQL_DATABASE	생성할 DataBase 이름
MYSQL_USER	스프링에서 사용할 계정의 아이디 정보
MYSQL_PASSWORD	스프링에서 사용할 계정의 DB 비밀번호
--net=spring-mysql-net	스프링 컨테이너와 통신하기 위한 네트워크

≡ Spring Boot 영역

IntelliJ의 Spring Boot 영역에 처리해야 할 내용입니다.

관련 작업 내역

파일	설명
WebConfigWindows.java	/images/ URL 요청을 컨테이너 내부 /images/ 폴더로 연결해 주는 역할(윈도우 용)
WebConfigLinux.java	/images/ URL 요청을 컨테이너 내부 /images/ 폴더로 연결해 주는 역할(리눅스 용)
application.properties	Spring Boot에서 어떤 환경(Profile)을 활성화할지를 지정하는 설정 spring.profiles.active= windows 항목 작성하기
pom.xml	<build> <finalName>docker_spring_mysql</finalName> </build>

```
cd
cd docker/shopping_03_mini_database/

tree -P "*.java" --prune # 확장자가 java인 파일만 확인하기
vi src/main/java/com/shopping/config/WebConfigLinux.java
vi src/main/java/com/shopping/config/WebConfigWindows.java
vi src/main/resources/application.properties
```

이 부분은 컨테이너 생성시 처리될 예정입니다.
작업할 내용이 없습니다.

≡ Spring Boot에서 JAR 파일 생성

사용자 정의 SpringBoot 이미지와 Mysql 이미지를 사용하여 간단한 웹 서버를 구현해 봅니다.

pom.xml 파일이 있는 곳에서 다음 명령어를 실행합니다.

-DskipTests 반드시 넣어 주도록 합니다.

```
mvn clean package -DskipTests
```

성공하게 되면 target 폴더에 docker_spring_mysql.jar 파일이 생성되어 있습니다.

```
ls target/docker_spring_mysql.jar
```

```
target/docker_spring_mysql.jar
```

≡ Dockerfile 만들기

pom.xml 파일이 존재하는 폴더의 위치에서 다음과 같은 파일을 작성합니다.

```
# 파일 이름 : Dockerfile01(Dockerfile의 이름은 개발자가 임의로 지정할 수 있습니다.)
# Temurin은 Eclipse 재단의 OpenJDK(Java 개발 키트) 무료 배포판으로 JDK가 포함(JRE + 컴파일 도구)되어 있습니다.
FROM eclipse-temurin:17-jdk

# Maven의 JAR 파일 위치 (target/*.jar) : Docker 이미지 빌드 시점에 전달 받을 매개 변수로 target/폴더 아래 모든 jar 파일을 의미합니다.
ARG JAR_FILE=target/*.jar

# 호스트 PC의 JAR 파일을 컨테이너 내부로 docker_spring_mysql.jar라는 이름으로 복사합니다.
# 기호 ${ }는 "변수 치환(variable expansion)" 문법입니다.
COPY ${JAR_FILE} docker_spring_mysql.jar

# 컨테이너 내부에서 사용할 네트워크 포트를 9000으로 지정합니다.
EXPOSE 9000

# 스프링 부트 애플리케이션 실행
# 컨테이너가 시작될 때 java -jar /docker_spring_mysql.jar라는 명령어를 자동 실행함을 의미합니다.
# 컨테이너도 하나의 프로세스인데 시작(진입)할 때, 자동으로 실행되는 명령어가 ENTRYPOINT입니다.
ENTRYPOINT ["java", "-jar", "/docker_spring_mysql.jar"]
```

≡ 이미지 빌드

SpringBoot로 만든 소스 코드를 나만의 사용자 정의 이미지로 만드는 과정입니다.

도커 이미지 빌드

주의 사항) 마지막 점 기호 누락하지 말 것(현재 디렉토리를 의미하는 build context)

```
docker build -t myspring-img -f Dockerfile01 .
```

도커 이미지가 생성이 되고 나면 docker.desktop의 Images 영역에 myspring-img라는 항목이 보입니다.

```
docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
httpd:latest	b913eada2685	177MB	47.6MB	
myspring-img:latest	12359c367afa	739MB	247MB	
mysql:latest	fe036967257b	1.29GB	293MB	U
nginx:latest	fb01117203ff	228MB	62.6MB	

≡ Spring Boot 컨테이너 생성

내가 직접 만든 Spring Boot 이미지를 사용하여 컨테이너를 생성하고 실행합니다.

Window

Spring Boot 컨테이너 생성

```
docker container run --name=spring-ctr -di --net=spring-mysql-net -v C:/boot/images:/images -p 9000:9000 -e  
SPRING_PROFILES_ACTIVE=linux -e SPRING_DATASOURCE_URL=jdbc:mysql://mysql-ctr:3306/coffee -e  
SPRING_DATASOURCE_USERNAME=gomdori -e SPRING_DATASOURCE_PASSWORD=Spring@1234 myspring-img  
d836d6db01e6019182aaf3a1e14136fd5b907d5595e77c8768c6b4661afebf1d
```

≡ Spring Boot 컨테이너 생성

내가 직접 만든 Spring Boot 이미지를 사용하여 컨테이너를 생성하고 실행합니다.

Linux

이미지 디렉토리 생성

cd

mkdir images

cp -r

~/docker/shopping_03_mini_database/src/main/resources/static
/images/. ~/images

Spring Boot 컨테이너 생성

-d # detached (백그라운드 실행)

-i # interactive (표준 입력 유지)

linux는 Spring의 @Profile 어노테이션 이름

docker container run \

--name spring-ctr \

-d -i \

--net spring-mysql-net \

-v ~/images:/images \

-p 9000:9000 \

-e SPRING_PROFILES_ACTIVE=linux \

-e SPRING_DATASOURCE_URL=jdbc:mysql://mysql-ctr:3306/coffee \

-e SPRING_DATASOURCE_USERNAME=gomdori \

-e SPRING_DATASOURCE_PASSWORD=Spring@1234 \

myspring-img

≡ DockerFile 응용

≡ Spring Boot 컨테이너 생성

내가 직접 만든 Spring Boot 이미지를 사용하여 컨테이너를 생성하고 실행합니다.

반영 여부 확인

스프링 컨테이너로 진입

```
docker exec -it spring-ctr bash
```

```
ls *.jar # jar 파일 있는 지 확인
```

```
# docker_spring_mysql.jar
```

```
ls /images/ # 이미지 보이는 지 확인
```

```
# 예시) cappuccino01.png 등등의 이미지가 있는 지 확인
```

```
exit # 빠져 나오기
```

≡ 테이블 생성

MySQL 컨테이너에 접속하여 테이블을 생성하도록 합니다.

```
docker exec -it mysql-ctr /bin/bash
mysql -u gomdori -p --default-character-set=utf8mb4
# docker exec -it -e LANG=en_US.UTF-8 -e LC_ALL=en_US.UTF-8 mysql-ctr mysql -u gomdori -p --default-character-set=utf8mb4
Enter password: Spring@1234

# SHOW VARIABLES LIKE 'character_set%';
# SHOW VARIABLES LIKE 'collation%';
USE coffee;

# 만약 테이블 생성이 안 되어 있는 경우 다음 구문을 사용하여 개발자가 생성하도록 합니다.
CREATE TABLE product (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255),
    price INT NOT NULL,
    description VARCHAR(255),
    image_url VARCHAR(255)
);
```


≡ 테이블 생성

MySQL 컨테이너에 접속하여 테이블을 생성하도록 합니다.

```
INSERT INTO product (name, price, description, image_url) VALUES
('아메리카노', 4444, '진한 에스프레소에 뜨거운 물을 더한 기본 블랙 커피', '/images/americano01.png'),
('브리오슈', 5555, '버터 풍미가 풍부하고 부드러운 식감의 프랑스식 달콤한 빵', '/images/brioche_02.png'),
('카푸치노', 6666, '에스프레소에 따뜻한 우유와 풍부한 우유 거품을 더한 클래식 커피', '/images/cappuccino01.png'),
('크로아상', 7777, '겉껍이 버터층이 살아 있는 바삭하고 고소한 프랑스식 페이스트리', '/images/croissant_01.png'),
('치아바타', 8888, '겉은 바삭하고 속은 촉촉한 이탈리아 전통 빵으로 샌드위치에 잘 어울림', '/images/ciabatta_07.png'),
('화이트 와인', 9999, '산뜻하고 과일향이 풍부한 화이트 와인', '/images/wine01.png');

commit ;

select * from product ;
```

≡ 테스트 하기

MySQL과 Spring Boot 컨테이너가 정상적으로 실행 중인지 확인합니다.

먼저 MySQL과 Spring Boot 컨테이너가 정상적으로 실행 중인지 확인합니다.

mysql-ctr와 spring-ctr가 STATUS: Up 상태여야 합니다.

포트 매핑도 확인 (9000:9000)

`docker container ps`

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					
44c9afe0c6f5	myspring-img	"java -jar /docker_s..."	3 minutes ago	Up 3 minutes	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
spring-ctr					
c7bbd6bffa39	mysql	"docker-entrypoint.s..."	30 minutes ago	Up 30 minutes	3306/tcp, 33060/tcp
mysql-ctr					

Spring Boot 컨테이너 로그 확인

Spring Boot 애플리케이션이 제대로 실행되었는지 확인하려면 Started Application in ... seconds 메시지가 나오면 정상입니다.

DB 연결 문제(Caused by: ...)가 없는지 확인

예시) Started ShoppingApplication in 9.728 seconds (process running for 11.193)

`docker logs -f spring-ctr`

≡ 테스트 하기

MySQL과 Spring Boot 컨테이너가 정상적으로 실행 중인지 확인합니다.

브라우저에서 다음 주소로 접속하여 테스트를 진행합니다.

호스트(Windows PC) 9000번 포트 → 컨테이너 9000번 포트 매핑

<http://xx.xx.xx.xx:9000/>

상품 목록



4,444원



5,555원



6,666원



7,777원



8,888원



9,999원

≡ 테스트 하기

다음 실습을 위하여 컨테이너를 종료하고, 삭제하도록 합니다.

```
docker container ls
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
44c9afe0c6f5	myspring-img	"java -jar /d..."	19 minutes ago	Up 19 minutes	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp	spring-ctr
c7bbd6bffa39	mysql	"docker-entry..."	46 minutes ago	Up 46 minutes	3306/tcp, 33060/tcp	mysql-ctr

```
docker container stop spring-ctr mysql-ctr
```

```
spring-ctr
```

```
mysql-ctr
```

```
docker container rm spring-ctr mysql-ctr
```

```
spring-ctr
```

```
mysql-ctr
```

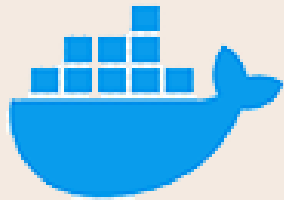
```
docker container ls
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

목록에 보이지 않아야 합니다.

Docker&Kubernetes

도커 컴포즈 사용하기

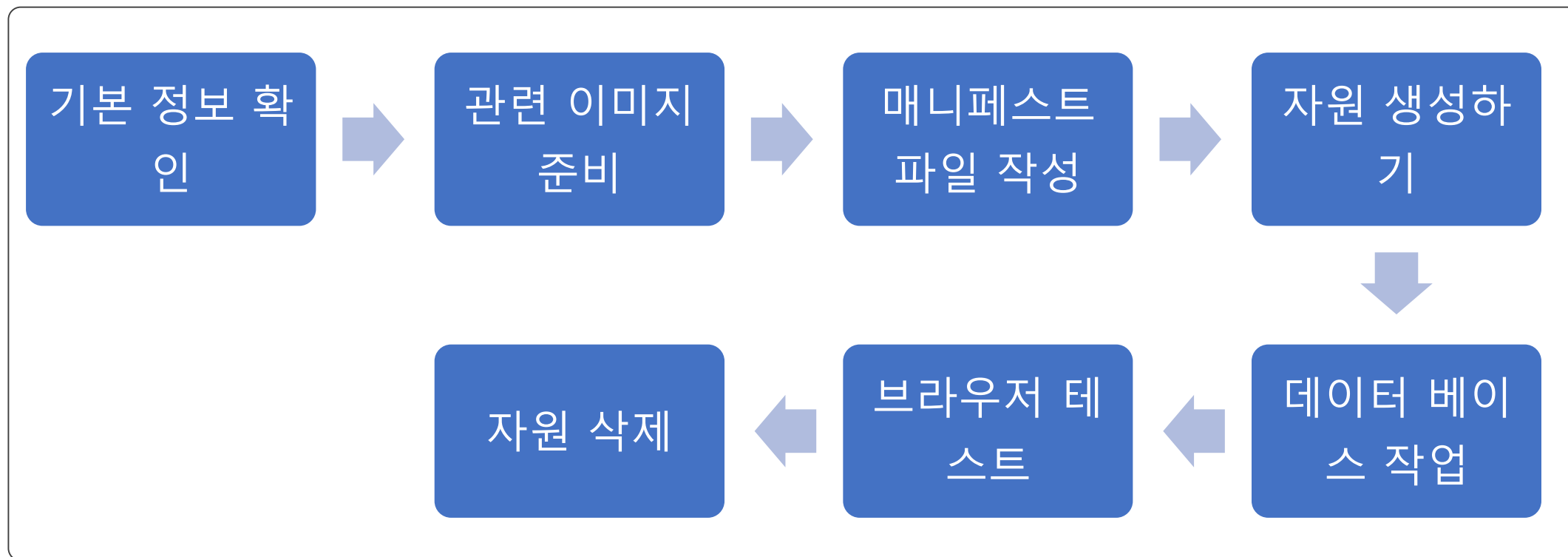


☰ Scenario01. 도커 컴포즈 사용하기

도커 컴포즈 파일을 이용하여 **MySQL**과 **SpringBoot 컨테이너**를 만들어 보겠습니다.

docker compose 실습하기(mysql+springboot).txt

실습 순서



☰ Scenario01. 기본 정보 확인

Step01. 도커 컴포즈를 작성하기 위한 기본 정보를 정의하도록 합니다.

컴포즈 정보

항목	값
네트워크 이름	spring-mysql-net
MySQL 볼륨 이름	mysql-spring-vol
MySQL 서비스 이름	mysql-svc
SpringBoot 서비스 이름	spring-svc

☰ Scenario01. 관련 이미지 준비

Step02. SpringBoot로 만든 소스 코드를 나만의 사용자 정의 이미지로 만드는 과정입니다.

이미지 준비

스프링 부트를 사용하여 미리 만들어 놓은 myspring-img을 준비합니다.

myspring-img는 docker build 명령어로 만들어 놓은 docker image의 이름입니다.

myspring-img 이미지가 생성되어 있는 지를 우선 확인합니다.

```
docker image ls
```

이전에 myspring-img 이미지를 미리 생성해 두었다면 아래 구문은 실행하지 않아도 됩니다.

도커 이미지 빌딩 작업을 위하여 Dockerfile 파일이 존재하는 디렉토리로 이동하여 실습을 진행합니다.

```
cd D:\docker\shopping_03_mini_database
```

```
docker build -t myspring-img -f Dockerfile01 .
```

도커 이미지가 생성이 되고 나면 docker.desktop의 Images 영역에 myspring-img라는 항목이 보입니다.

☰ Scenario01. 매니페스트 파일 작성

Step03. 환경의 전체 설계도(specification)를 의미하는 파일을 작성합니다.

파일 위치

항목	값
Windows	D:\docker\compose01\compose_mysql_springboot.yml
Linux	~/docker/compose01/compose_mysql_springboot.yml

Scenario01. 매니페스트 파일 작성

Step03. 매니페스트 파일에 MySql 컨테이너에 대한 정보를 작성합니다.

MySql 컨테이너 정보

항목	항목 이름	값
MySql 이미지 이름	image:	mysql:8.0
사용할 네트워크	networks:	spring-mysql-net
사용할 볼륨	volumes:	mysql-spring-vol
마운드 할 위치		/var/lib/mysql
재시작 설정	restart:	always
MySQL 기본 문자 집합과 정렬 규칙	command:	--character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
MySql 설정	environment:	하단 ☉이 붙은 항목 설정
☉ MySql 루트 비밀번호	MYSQL_ROOT_PASSWORD	My@Sql01
☉ MySql 데이터 베이스 이름	MYSQL_DATABASE	coffee
☉ MySql 사용자 이름	MYSQL_USER	gomdori
☉ MySql 비밀번호	MYSQL_PASSWORD	Spring@1234

Scenario01. 매니페스트 파일 작성

Step03. 매니페스트 파일에 Spring Boot 컨테이너에 대한 정보를 작성합니다.

스프링 부트 컨테이너 정보

MySQL 컨테이너 관련 코드를 먼저 작성하고, SpringBoot 컨테이너는 의존 관계를 고려하여 depends_on 항목을 추가하도록 합니다.

항목	항목 이름	값
의존 관계	depends_on:	mysql-svc
이미지 이름	image:	myspring-img
사용할 네트워크	networks:	spring-mysql-net
포트 번호 설정	ports:	9000:9000
재시작 설정	restart:	always
데이터 베이스 관련 정보	environment:	하단 ☉이 붙은 항목 설정
☉ 스프링의 활성 프로파일 이름	SPRING_PROFILES_ACTIVE:	docker
☉ 데이터 베이스 접속 URL	SPRING_DATASOURCE_URL:	jdbc:mysql://mysql-svc:3306/coffee
☉ 데이터 베이스 사용자 이름	SPRING_DATASOURCE_USERNAME:	gomdori
☉ 데이터 베이스 비밀번호	SPRING_DATASOURCE_PASSWORD:	Spring@1234

≡ Scenario01. 자원 생성하기

Step04. 작성된 도커 컴포즈 파일을 사용하여 이를 실행해 보도록 합니다.

docker-compose up 명령어를 실행하면 컴포즈 파일의 정의대로 컨테이너 및 주변 환경이 생성됩니다.
-p 옵션 : 컨테이너 이름 지정, -f 옵션 : 컴포즈 파일의 경로 지정, -d 옵션은 백그라운드에서 사용함을 의미합니다.

Window

```
cd \docker
```

```
# docker compose -p 생성할컨테이너이름 -f 매니페스트파일의경로와이름 up -d
```

```
docker compose -p hello -f d:\docker\compose01\compose_mysql_springboot.yml up -d
```

☰ Scenario01. 자원 생성하기

Step04. 작성된 도커 컴포즈 파일을 사용하여 이를 실행해 보도록 합니다.

docker-compose up 명령어를 실행하면 컴포즈 파일의 정의대로 컨테이너 및 주변 환경이 생성됩니다.
-p 옵션 : 컨테이너 이름 지정, -f 옵션 : 컴포즈 파일의 경로 지정, -d 옵션은 백그라운드에서 사용함을 의미합니다.

Linux

```
cd
docker compose -p hello -f ~/docker/compose01/compose_mysql_springboot.yml up -d
[+] up 17/17
✓ Image mysql:8.0          Pulled                27.0s
✓ Network hello_spring-mysql-net Created           0.1s
✓ Volume hello_mysql-spring-vol Created            0.0s
✓ Container mysql-ctr      Created                 8.7s
✓ Container spring-ctr     Created

# 컨테이너 이름과 상태만 확인
docker container ps -a --format "table {{.Names}}\t{{.Status}}"
NAMES      STATUS
spring-ctr Up 55 seconds
mysql-ctr  Up About a minute
```

☰ Scenario01. 데이터 베이스 작업

Step05. 생성된 MySql 컨테이너에 접속하여 목록 페이지의 제목을 변경해 진행합니다.

데이터 베이스 접속하여 다음과 같이 실습을 진행합니다.

```
docker exec -it mysql-ctr /bin/bash
```

-p 옵션과 비밀번호를 붙여서 입력합니다.

```
mysql -u spring -p
```

```
Enter Password : Spring@1234
```

다음과 같이 작성하여 테이블에 데이터를 추가하도록 합니다.

```
USE coffee;
```

```
INSERT INTO product (name, price, description, image_url) VALUES  
(`americano01`, 4444, `americano01 coffee`, `/images/americano01.png`),  
(`brioche_02`, 5555, `brioche_02 빵`, `/images/brioche_02.png`),  
(`cappuccino01`, 6666, `cappuccino01 커피`, `/images/cappuccino01.png`),  
(`croissant_01`, 7777, `croissant_01 페이스트리`, `/images/croissant_01.png`),  
(`ciabatta_07`, 8888, `ciabatta_07 빵`, `/images/ciabatta_07.png`),  
(`wine01`, 9999, `wine01 화이트 와인`, `/images/wine01.png`);
```

```
commit ;
```

```
select * from product ;
```

```
exit # 터미널 창으로 빠져 나가기
```

```
exit ; # 컨테이너 빠져 나가기
```

☰ Scenario01. 브라우저 테스트

Step06. 브라우저에 다음 주소를 입력하여 다음과 같이 동일하게 출력이 되는지 점검합니다.

혹시 이미지가 보이지 않으면 다음 문장을 실행합니다.

```
docker cp ~/images spring-ctr:/
```

브라우저에서 확인

<http://localhost:9000> # Nat 모드

<http://xx.xx.xx.xx:9000> # Bridge 모드

상품 목록



americano01
4,444원



brioche_02
5,555원



cappuccino01
6,666원



croissant_01
7,777원



ciabatta_07
8,888원



wine01
9,999원

≡ Scenario01. 자원 삭제

Step07. down 명령어를 사용하여 컨테이너와 네트워크를 종료한 다음 삭제하도록 합니다.

```
# 컴포즈 파일로 생성된 관련 컨테이너 등을 삭제해 보도록 합니다.  
# down 명령어를 사용하여 컨테이너와 네트워크를 종료한 다음 삭제하도록 합니다.  
# Windows 실습  
docker compose -p hello -f D:\docker\compose01\compose_mysql_springboot.yml down  
  
# Linux 실습  
docker compose -p hello -f ~/docker/compose01/compose_mysql_springboot.yml down  
  
# 이미지 목록 확인 : down 명령어를 사용하더라도, 이미지와 볼륨 정보는 삭제되지 않습니다.  
docker images ls  
  
# 이미지 정보를 삭제합니다.  
docker image rm mysql:8.0  
docker image rm myspring-img  
  
# 볼륨 정보를 삭제합니다.  
docker volume rm hello_mysql-spring-vol
```


☰ Scenario02. 도커 컴포즈 사용하기

도커 컴포즈 파일을 이용하여 **MySQL**과 **WordPress 컨테이너**를 만들어 보겠습니다.

MySQL 컨테이너 관련 코드를 먼저 작성하고, WordPress 컨테이너는 의존 관계를 고려하여 depends_on 항목을 추가하도록 합니다.

컴포즈 파일 배치 경로

항목	값
컴포즈 파일 위치	D:\docker\compose02\compose_mysql_wordpress.yml

주요 항목 작성



이름 작성



MySQL 컨테이너 정의



WordPress 컨테이너 정의

☰ Scenario02. 도커 컴포즈 사용하기

도커 컴포즈 파일을 이용하여 MySql과 WordPress 컨테이너를 만들어 보겠습니다.

컴포즈 정보

항목	값
네트워크 이름	network01
MySql 볼륨 이름	volume_mysql
WordPress 볼륨 이름	volume_wordpress
MySql 컨테이너 이름	mysql01
WordPress 컨테이너 이름	wordpress01

☰ Scenario02. 도커 컴포즈 사용하기

도커 컴포즈 파일을 이용하여 MySql과 WordPress 컨테이너를 만들어 보겠습니다.

MySql 컨테이너정보

항목	항목 이름	값
MySql 이미지 이름	image:	mysql:8.0
사용할 네트워크	networks:	network01
사용할 볼륨	volumes:	- volume_mysql
마운트 할 위치		/var/lib/mysql
재시작 설정	restart:	always
MySQL 기본 문자 집합과 정렬 규칙	command:	--character-set-server=utf8mb4 --collation-server=utf8mb4_unicode_ci
MySql 설정	environment:	하단 ☉이 붙은 항목 설정
☉ MySql 루트 비밀번호	MYSQL_ROOT_PASSWORD	root
☉ MySql 데이터 베이스 이름	MYSQL_DATABASE	coffee
☉ MySql 사용자 이름	MYSQL_USER	root
☉ MySql 비밀번호	MYSQL_PASSWORD	My@Sql01

☰ Scenario02. 도커 컴포즈 사용하기

도커 컴포즈 파일을 이용하여 MySql과 WordPress 컨테이너를 만들어 보겠습니다.

워드 프레스 컨테이너정보

항목	항목 이름	값
의존 관계	depends_on:	- mysql01
워드프레스 이미지 이름	image:	wordpress
사용할 네트워크	networks:	- networks01
사용할 볼륨	volumes:	- volume_wordpress
마운트 위치		/var/www/html
포트 번호 설정	ports:	- 8085:80
재시작 설정	restart:	always
데이터 베이스 관련 정보	environment:	하단 ◎이 붙은 항목 설정
◎ 데이터 베이스 컨테이너 이름	WORDPRESS_DB_HOST:	mysql01
◎ 데이터 베이스 이름	WORDPRESS_DB_NAME:	coffee
◎ 데이터 베이스 사용자 이름	WORDPRESS_DB_USER:	root
◎ 데이터 베이스 비밀번호	WORDPRESS_DB_PASSWORD:	My@Sql01

☰ Scenario02. 도커 컴포즈 사용하기

작성된 도커 컴포즈 파일을 사용하여 이를 실행해 보도록 합니다.

docker-compose up 명령어를 실행하면 컴포즈 파일의 정의대로 컨테이너 및 주변 환경이 생성됩니다.

-d 옵션을 사용하고, -f 옵션을 사용하여 컴포드 파일의 경로를 지정하도록 합니다.

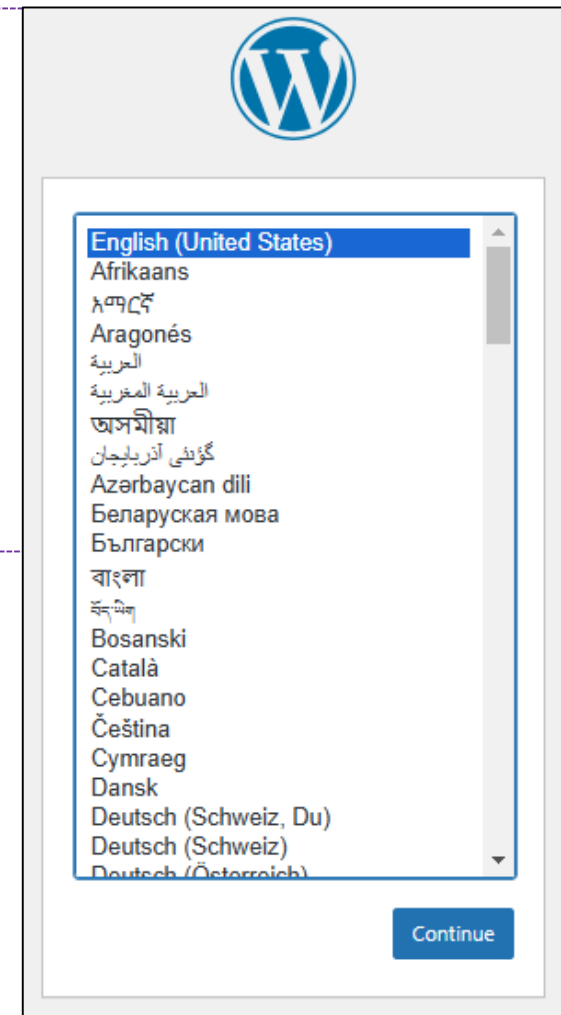
```
docker compose -f D:\docker\compose02\compose_mysql_wordpress.yml up -d
```

브라우저에 다음 주소를 입력하여 오른쪽 화면과 같이 동일하게 출력이 되는지 점검합니다

<http://localhost:8085>

down 명령어를 사용하여 컨테이너와 네트워크를 종료한 다음 삭제하도록 합니다.

```
docker compose -f D:\docker\compose02\compose_mysql_wordpress.yml down
```



☰ Scenario02. 도커 컴포즈 사용하기

컴포즈 파일로 생성된 관련 컨테이너 등을 삭제해 보도록 합니다.

down 명령어를 사용하더라도, 이미지와 볼륨 정보는 삭제되지 않습니다.

이미지 정보를 삭제합니다.

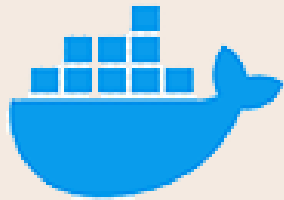
```
docker image rm mysql wordpress
```

볼륨 정보를 삭제합니다.

```
docker volume rm windows_volume_mysql windows_volume_wordpress
```

Docker&Kubernetes

리눅스에 도커 설치



Docker 설정

Ubuntu 가상 머신에 Docker를 위한 설정 작업을 합니다.

참조 파일 : docker_setup.sh.txt

Ubuntu에 Docker 설정

도커 레지스트리와 통신할 수 있는 필요한 패키지들을 설치합니다.

```
sudo cp docker_setup.sh.txt /root/docker_setup.sh
```

```
sudo su -
```

```
apt update
```

dos2unix : Windows에서 만든 쉘 스크립트를 Linux에서 실행해서 생기는 CRLF 문제 해결책

```
apt install dos2unix -y
```

```
dos2unix docker_setup.sh
```

```
chmod +x docker_setup.sh
```

```
./docker_setup.sh
```

다음 명령어를 실행하여 결과와 같이 출력이 되면 성공한 것입니다.

```
docker container --help
```


리눅스에 도커 설치

Docker 설정

Ubuntu 가상 머신에 Docker를 위한 설정 작업을 합니다.

Ubuntu에 Docker 설정

```
systemctl status docker
```

위의 결과에서 active(running)라는 메시지가 보이면 현재 활성 상태입니다.

Ctrl + C를 눌러서 빠져 나갑니다.

다음 명령어를 사용하여 일반 사용자 모드로 이동합니다.

```
exit
```

```
sudo usermod -aG docker $USER
```

groups 명령어를 사용하여 현재 사용자(\$USER)가 docker 그룹에 속해 있는지 확인합니다.

```
groups $USER
```

bluesky : bluesky adm cdrom sudo dip plugdev lxd **docker** #
현재 사용자(bluesky)는 docker 그룹에 속해 있음을 알 수 있습니다.

도커 버전 확인 : 혹시 "permission denied"이라는 메시지를 보게 되면 logout을 수행 후 login 후 테스트 해보세요.

```
docker version
```

```
logout
```

```
docker version
```

 # 재로그인 후 도커 버전 확인

Client: Docker Engine - Community

Version: 29.1.3

API version: 1.52

... 중략

Server: Docker Engine - Community

Engine:

Version: 29.1.3

API version: 1.52(minimum version 1.44)

... 중략

리눅스에 도커 설치

작업 순서

다음과 같은 작업 순서로 설치를 진행합니다.

구현 절차

연관 패키지 설치

GPG 키 등록

저장소 추가

```
sudo apt update
```

```
sudo apt install docker-ce docker-ce-cli containerd.io
```

리눅스에 도커 설치

연관 패키지 설치

다음과 같이 설치 관련하여 패키지 업데이트 및 항목들을 다운로드 합니다.

구현하기

AWS의 EC2 또는 OCI의 Instance에서 실행하면 됩니다.

```
# 도커 레지스트리와 통신할 수 있는 필요한 패키지들을 설치합니다.
```

```
sudo su -
```

```
# 시스템의 패키지 목록을 최신 상태로 업데이트합니다.
```

```
apt update
```

```
apt install -y apt-transport-https
```

```
apt install -y ca-certificates curl software-properties-common
```

```
apt install -y gnupg lsb-release
```

리눅스에 도커 설치

GPG 키 등록

GPG는 GnuPG(GNU Privacy Guard)로 암호화 및 인증에 사용되는 도구입니다.

```
# Docker의 공식 GPG 키를 시스템에 추가합니다.
# GnuPG(GNU Privacy Guard)는 데이터를 암호화하고 서명하는 데 사용되는 무료 오픈 소스 소프트웨어입니다.
# GnuPG는 PGP(Pretty Good Privacy)와 호환되며, 안전한 통신과 데이터 저장을 위해 설계되었습니다.
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# 실행하면 프롬프트에는 아무런 변화가 없습니다.
# --dearmor: GPG 키를 바이너리 형식으로 변환합니다.
# 일반적으로 GPG 키는 ASCII 형식이지만, 시스템에서 효율적으로 처리하기 위해 바이너리 형식으로 변환합니다.

# -o /usr/share/keyrings/docker-archive-keyring.gpg:
# -o: 출력 파일 경로를 지정합니다.
# /usr/share/keyrings/docker-archive-keyring.gpg: 변환된 바이너리 GPG 키를 저장할 위치입니다.
# 이 파일은 Docker 저장소 인증에 사용됩니다.

# 다운로드가 잘 되었는지 확인합니다.
ls -al /usr/share/keyrings/d*

-rw-r--r-- 1 root root 2760 Apr 17 03:42 /usr/share/keyrings/docker-archive-keyring.gpg
```

저장소 추가

Docker의 안정 버전 저장소를 다음과 같이 추가합니다.

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu  
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

실행하면 프롬프트에는 아무런 변화가 없습니다.

명령어 요약

항목	설명
echo "deb [...]"	Docker 저장소 정보를 한 줄로 생성
\$(dpkg --print-architecture)	현재 시스템의 CPU 아키텍처 자동 반영
\$(lsb_release -cs)	Ubuntu 버전 코드네임 자동 입력
sudo tee ...	관리자 권한으로 APT 설정 파일 작성
> /dev/null	출력 억제 (화면에 아무것도 안 보이게 함)

Docker 설치

저장소를 업데이트한 후 Docker를 설치합니다.

업데이트하지 않고, 다음 과정을 거치면 패키지의 최신 버전을 찾지 못하여 설치 도중 에러가 발생할 수 있습니다.

아래의 update 구문 무조건 실행하도록 합니다.

```
apt update
```

```
apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

설치 도중에 'yes' 또는 'no'를 입력할 수 있는 데 'y'를 입력하면 됩니다.

완료하는 5~10분 정도의 시간이 소요됩니다.

리눅스에 도커 설치

Docker 설치 여부 확인

도커가 잘 설치되었는지 확인하기 위하여 다음 명령어를 입력해 보세요.

다음 명령어를 실행하여 결과와 같이 출력이 되면 성공한 것입니다.

`docker container --help`

Usage: docker container COMMAND

#

Manage containers

#

Commands:

attach Attach local standard input, output, and error streams to a running container

commit Create a new image from a container's changes

cp Copy files/folders between a container and the local filesystem

... 중략

wait Block until one or more containers stop, then print their exit codes

#

Run 'docker container COMMAND --help' for more information on a command.

리눅스에 도커 설치

Docker 활성화 여부 확인

다음 명령어를 사용하여 현재 도커가 활성화 중인지 체크합니다.

```
systemctl status docker
```

```
# docker.service - Docker Application Container Engine
#    Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
#    Active: active (running) since Thu 2024-12-12 11:46:39 UTC; 3min 41s ago
# TriggeredBy: ● docker.socket
#    Docs: https://docs.docker.com

# 위의 결과에서 active (running)라는 메시지가 보이면 현재 활성 상태입니다.
# Ctrl + C를 눌러서 빠져 나갑니다.
```


리눅스에 도커 설치

Docker 사용 권한 부여

현재 사용자가 `sudo` 명령어 없이 `docker` 명령을 사용할 수 있도록 `docker` 그룹에 사용자를 추가합니다.

다음 명령어를 사용하여 일반 사용자 모드로 이동합니다.

`exit`

`usermod` 명령어는 사용자 계정의 속성을 수정하는 데 사용되는 명령어입니다.

변수 `'USER'`는 현재 사용자의 `id` 이름을 의미하고, `-aG` 옵션은 사용자에게 추가 그룹을 지정해주는 옵션입니다.

`sudo usermod -aG docker $USER`

`groups` 명령어를 사용하여 현재 사용자(`$USER`)가 `docker` 그룹에 속해 있는지 확인합니다.

`groups $USER`

`bluesky : bluesky adm cdrom sudo dip plugdev lxd docker` # 현재 사용자(`bluesky`)는 `docker` 그룹에 속해 있음을 알 수 있습니다.

리눅스에 도커 설치

Docker 사용 권한 부여

일반 사용자로 다시 docker에 대한 버전 정보를 확인해 봅니다.

도커 버전 확인 : 혹시 "permission denied"이라는 메시지를 보게 되면 logout을 수행 후 login 후 테스트 해보세요.

`docker version`

`logout`

`docker version` # 재로그인 후 도커 버전 확인

Client: Docker Engine - Community

Version: 29.1.3

API version: 1.52

... 중략

Server: Docker Engine - Community

Engine:

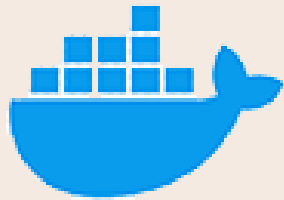
Version: 29.1.3

API version: 1.52 (minimum version 1.44)

... 중략

Docker&Kubernetes

쿠버네티스 실습



쿠버네티스 실습

쿠버네티스 실습

실습 파일

쿠버네티스 실습.txt